MÁSTER EN ANÁLISIS DE MALWARE Y REVERSING

MÓDULO 7. DEBUG Y ANÁLISIS DEL SISTEMA OPERATIVO (LINUX)

ANÁLISIS Y DEPURACIÓN DEL SO

MÁSTER EN *ANÁLISIS DE MALWARE Y REVERSING*

Campus Internacional CIBERSEGURIDAD

ENIIT INNOVA IT BUSINESS SCHOOL

UCAM UNIVERSIDAD CATÓLICA DE MURCIA

INDICE DE CONTENIDOS

1. Recolección de información 4
1.1. Recolección de métricas 4
1.2. Estado general de la máquina 7
1.2.1. Obteniendo información del SO 7
1.2.2. Listado del hardware 8
1.2.3. Identificando el proveedor del servicio y datos relativos a la suscripción ... 14
1.3. Recolección de logs 19
2. El directorio /proc 21
2.1. Organización y estructura del directorio /proc 22
2.2. Directorios virtuales y ficheros relacionados con el hardware 23
2.3. Directorios y ficheros virtuales que ofrecen información del sistema 27
2.4. Representación de procesos en /proc 30
2.5. Ficheros de escritura en /proc. Configurando el sistema 32
2.5.1. /proc/sys 33
3. ¿Qué expone la máquina a internet? 34
4. Uso de las llamadas del kernel que hacen las apps 40
4.1. Strace 40
4.1.1. Uso de strace 40
4.1.2. Filtrando llamadas concretas al sistema 41
4.1.3. Algunas syscall interesantes a tener en cuenta a la hora de usar strace 42
5. Reescritura de módulos del kernel para extraer información del sistema 43
5.1. Gestión de módulos del kernel 44
5.1.1. Listado de los módulos cargados en el kernel 44
5.1.2. Descarga de módulos del kernel 46
5.1.3. Carga de módulos del kernel 47
5.1.4. Configuración de opciones de los módulos del kernel 48
5.2. Escritura e instalación de un módulo del kernel 49
5.2.1. Escritura y compilación de nuestro módulo 49
5.2.2. Instalación de nuestro módulo en el kernel 50
5.2.3. Instalación/desinstalación de un módulo propio 52

6. Depuración de la actividad del kernel 53
6.1. La herramienta dmesg 53
6.2. Configuración de los niveles de log del kernel 53
6.3. Kernel Tainting. "Manchando" el kernel a propósito 56
6.4. Syscall Hooking. Como intervenir llamadas al Sistema 57
7. Customizando el kernel de Linux para depuración 58
7.1. Estructura del código fuente del Kernel de Linux 59
7.2. Obtención y customizado el código fuente del kernel 60
7.3. Compilación del kernel 62
7.3.1. Configuración del kernel: 63
7.3.2. Construyendo un kernel custom para depuración 70
7.3.3. Herramientas para la depuración del kernel – Kprobes 70
7.4. Modificación de la API del sistema operativo. Exfiltrando a través de glibc 71
8. Depuración de Crash/Core dumps y swap files/partitions 77
BIBLIOGRAFÍA 79

1. RECOLECCIÓN DE INFORMACIÓN

La base de una buena vigilancia de la infraestructura es **la obtención de información**. Esta debe ser obtenida de forma **correcta y precisa** ya que el correcto funcionamiento del resto de sistemas **depende de ello**.

1.1. RECOLECCIÓN DE MÉTRICAS

Métrica: Una métrica es un valor normalmente numérico y asociado a un momento exacto en el tiempo. Utilizando métricas, es posible por ejemplo generar gráficas, realizar agregaciones de datos, etc. Por ejemplo, pensemos en el uso de memoria RAM a lo largo del tiempo. Una métrica será cualquiera de las mediciones realizadas en cada momento de tiempo. La recolección de métricas rara vez es realizada de forma continua en el tiempo, si no que se va realizando periódicamente a intervalos de tiempo. Por ejemplo, comprobar la RAM cada 5 segundos. Esto se realiza para tratar de afectar lo estrictamente necesario al sistema a monitorizar. La correcta elección de los períodos de muestreo es fundamental para poder disponer de la información necesaria.

Existen numerosas soluciones específicas que ofrecen esto como servicio, simplemente instalándolas en cada instancia (ej: collectd, monit, ...)

La mayoría de estas soluciones ofrecen una serie de plugins desarrollados que cubren la mayor parte de necesidades de monitorización de los usuarios, pero además, ofrecen la posibilidad a los usuarios de incluir nuevos plugins que estos desarrollen para monitorizar sus propios recursos.

Ejemplos de recursos monitorizables.

- Usuarios
  - ¿Cuántos usuarios existen en el sistema?
  - ¿Cuántos usuarios hay actualmente trabajando en la máquina?
  - ¿Cuántos usuarios existen en la máquina de tal tipo?
  - ¿Cuántos grupos?
  - ...
- Conexiones
  - Conexiones abiertas

Campus Internacional CIBERSEGURIDAD
ENIIT INNOVA IT BUSINESS SCHOOL
UCAM UNIVERSIDAD CATÓLICA DE MURCIA

    - Conexiones abiertas para puertos específicos
    - Por protocolo
    - Tráfico de entrada / salida de red
    - ...
- CPU
    - Carga a largo, corto y medio plazo
    - % uso de CPU
    - tiempo de CPU medio de los procesos
    - ...
- RAM
    - Memoria libre, usada, intercambio, cacheada
    - Porcentaje de páginas sucias
    - ...
- Disco
    - Espacio utilizado, libre
    - Tasa de transferencia de escritura / lectura
    - Numero de ficheros abiertos
    - Numero de ficheros creados
- Otro Hardware y dispositivos
    - Información de los dispositivos
    - Estado frente al sistema (puntos de montaje, uso, etc.)
    - Consumidores y registros de actividad
    - Datos producidos en modo normal de operación
    - ...
- Procesos
    - Número de procesos (global o por usuario)
    - ¿Existe el proceso con PID en el sistema?
    - Porcentaje de procesos zombie, idle, running, etc.

Campus Internacional CIBERSEGURIDAD
ENIIT INNOVA IT BUSINESS SCHOOL
UCAM UNIVERSIDAD CATÓLICA DE MURCIA

  - Numero de procesos cuya prioridad es inferior o superior a
  - ...
- Operación del Kernel
  - Interrupiones y llamadas a funciones
  - Rutinas
  - Arranque y detención
  - Kernel Panic y errores no recuperables
  - ...
- Específicos a nivel de aplicación
  - Base de datos. Numero de tablas en una colección, Número de registros en una tabla, número de operaciones realizadas, etc.
  - Sistema de colas. Numero de mensajes por cola, tasa de inserción y consumo, ...
  - ...
- ...

Como se ha comentado antes, cualquier recurso es monitorizable. Para obtener métricas solo es necesario que podamos asociar un valor cuantificable sobre el mismo en función del tiempo.

Ejemplos de herramientas y comandos útiles.

- **htop**: Muestra estadísticas de uso de CPU, RAM, procesos, carga, swap, ...
- **bmon**: Estadísticas de uso de red.
- **pstree**: Muestra el árbol genealógico de los procesos. (Padres, hijos, abuelos, ...)
- **ps**: Muestra información de los procesos actuales.
- **free**: Recoge el uso de memoria.
- **netstat**: Recoge información sobre las conexiones de red, tablas de enrutado, estadísticas de uso de interfaces, máscaras de conexiones, etc.
- **iotop**: Muestra estadísticas de uso de disco
- **hwls:** Muestra información detallada sobre el hardware instalado en la máquina

Campus Internacional CIBERSEGURIDAD
ENIIT INNOVA IT BUSINESS SCHOOL
UCAM UNIVERSIDAD CATÓLICA DE MURCIA

Enlaces interesantes…

http://manpages.ubuntu.com/manpages/xenial/man1/htop.1.html

http://manpages.ubuntu.com/manpages/trusty/man8/bmon.8.html

http://manpages.ubuntu.com/manpages/artful/man1/pstree.1.html

http://manpages.ubuntu.com/manpages/xenial/man1/ps.1.html

http://manpages.ubuntu.com/manpages/xenial/es/man8/netstat.8.html

http://manpages.ubuntu.com/manpages/artful/man8/iotop.8.html

1.2. ESTADO GENERAL DE LA MÁQUINA

Uno de los escenarios en los que mayor sentido cobra este módulo es aquél en el que no conocemos nada de la máquina a la que acabamos de tener acceso, ya sea físico o remoto, por ejemplo, vía ssh.

En este escenario, una de las primeras tareas que deberían llevarse a cabo es obtener una visión global de la máquina, con el objetivo de conocer exactamente qué tenemos ante nosotros y qué herramientas y procedimientos podrán ser utilizados en cada caso para encontrar la información que buscamos. Este proceso, guarda cierta similitud con lo que realizaría un atacante que acaba de conseguir acceso a una máquina.

Así pues, en esta primera fase de reconocimiento, será útil disponer de características técnicas de la máquina, conocer si se trata de una máquina nativa o virtual, proveedor del servicio (si es virtual), versión del SO y kernel, su estado general (ocupación de disco, RAM, etc) y en definitiva cualquier información útil para acotar los trabajos de análisis que se realizarán posteriormente.

1.2.1. Obteniendo información del SO

Una de las opciones mas populares es el uso de **uname**, cuya salida por defecto muestra el nombre del sistema operativo, pero que puede ser ampliada con distintas opciones como por ejemplo **-n** (imprime el nombre de la máquina), **-v** (imprime la versión del SO) o **-a** que imprime una combinación bastante amplia de opciones que sería equivalente a **-mnrsv**

En el caso por ejemplo de Mac, obtendríamos una salida similar a la siguiente

```
➜ ~ uname
Darwin
➜ ~ uname -n
MacBook-Pro-de-Jhon-Doe.local
➜ ~ uname -v
Darwin Kernel Version 18.7.0: Tue Nov 10 00:07:31 PST 2020; root:xnu-4903.278.51~1/RELEASE_X86_64
➜ ~ uname -a
Darwin MacBook-Pro-de-Jhon-Doe.local 18.7.0 Darwin Kernel Version 18.7.0: Tue Nov 10 00:07:31 PST 2020; root:xnu-4903.278.51~1/RELEASE_X86_64 x86_64
```

Y para Ubuntu:

```
admin1@mru8:~$ uname -a
Linux mru8 4.15.0-88-generic #88-Ubuntu SMP Tue Feb 11 20:11:34 UTC 2020 x86_64 x86_64 x86_64 GNU/Li
nux
admin1@mru8:~$ uname
Linux
admin1@mru8:~$ uname -n
mru8
admin1@mru8:~$ uname -v
#88-Ubuntu SMP Tue Feb 11 20:11:34 UTC 2020
admin1@mru8:~$ uname -a
Linux mru8 4.15.0-88-generic #88-Ubuntu SMP Tue Feb 11 20:11:34 UTC 2020 x86_64 x86_64 x86_64 GNU/Li
nux
```

Como se puede observar estamos ejecutando Ubuntu sobre la versión 4.15.0 del kernel.

1.2.2. Listado del hardware

El comando **lshw** puede ofrecernos una información mas que detallada de las prestaciones, HW añadido, conexiones, etc. que incluye la máquina. Debe ser lanzado como sudo y podemos obtenerse un resumen de toda la información usando la opción **-short**

La salida que obtendríamos sería similar a la siguiente (extendida):

```
mru8
    description: Computer
    product: VMware Virtual Platform
    vendor: VMware, Inc.
    version: None
    serial: VMware-56 4d 89 a4 b4 85 e8 42-6d 47 10 c0 62 f5 1f 7d
    width: 64 bits
    capabilities: smbios-2.7 dmi-2.7 vsyscall32
    configuration: administrator_password=enabled boot=normal frontpanel_password=unknown keyboard_password=unknow
89A4-B485-E842-6D47-10C062F51F7D
  *-core
       description: Motherboard
       product: 440BX Desktop Reference Platform
       vendor: Intel Corporation
       physical id: 0
       version: None
       serial: None
     *-firmware
          description: BIOS
          vendor: Phoenix Technologies LTD
          physical id: 0
          version: 6.00
          date: 05/19/2017
          size: 86KiB
          capabilities: isa pci pcmcia pnp apm upgrade shadowing escd cdboot bootselect edd int5printscreen int9ke
ideo acpi smartbattery biosbootspecification netboot
     *-cpu:0
          description: CPU
          product: Intel(R) Core(TM) i7-7660U CPU @ 2.50GHz
          vendor: Intel Corp.
          physical id: 4
          bus info: cpu@0
          version: Intel(R) Core(TM) i7-7660U CPU @ 2.50GHz
          slot: CPU #000
          size: 2500MHz
          capacity: 4230MHz
          width: 64 bits
          capabilities: x86-64 fpu fpu_exception wp vme de pse tsc msr pae mce cx8 apic sep mtrr pge mca cmov pat
```

Con la opción -short la salida sería la equivalente a la siguiente (tomada de una de las máquinas virtual del módulo)

```
admin1@mru8:~$ sudo lshw -short
H/W path        Device     Class        Description
==========================================================
                           system       VMware Virtual Platform
/0                         bus          440BX Desktop Reference Platform
/0/0                       memory       86KiB BIOS
/0/4                       processor    Intel(R) Core(TM) i7-7660U CPU @ 2.50GHz
/0/4/94                    memory       16KiB L1 cache
/0/5                       processor    CPU
/0/5/95                    memory       16KiB L1 cache
/0/6                       processor    CPU
/0/4a/da                   memory       16KiB L1 cache
```

```
...
...
/0/82/112                memory      16KiB L1 cache
/0/83                    processor   CPU
/0/83/113                memory      16KiB L1 cache
/0/1a2                   memory      962MiB System Memory
/0/100                   bridge      440BX/ZX/DX - 82443BX/ZX/DX Host bridge
/0/100/1                 bridge      440BX/ZX/DX - 82443BX/ZX/DX AGP bridge
/0/100/7                 bridge      82371AB/EB/MB PIIX4 ISA
/0/100/7.1               storage     82371AB/EB/MB PIIX4 IDE
/0/100/7.3               bridge      82371AB/EB/MB PIIX4 ACPI
/0/100/7.7               generic     Virtual Machine Communication Interface
/0/100/f                 display     SVGA II Adapter
/0/100/10        scsi32  storage     53c1030 PCI-X Fusion-MPT Dual Ultra320 SCSI
/0/100/10/0.0.0  /dev/sda  disk      21GB VMware Virtual S
/0/100/10/0.0.0/1  /dev/sda1  volume  1023KiB BIOS Boot partition
/0/100/10/0.0.0/2  /dev/sda2  volume  19GiB EXT4 volume
/0/100/11                bridge      PCI bridge
/0/100/11/0              bus         USB1.1 UHCI Controller
/0/100/11/0/1    usb2    bus         UHCI Host Controller
/0/100/11/0/1/1          input       VMware Virtual USB Mouse
/0/100/11/0/1/2          bus         VMware Virtual USB Hub
/0/100/11/0/1/2/1        communication  Virtual Bluetooth Adapter
/0/100/11/1      ens33   network     82545EM Gigabit Ethernet Controller (Copper)
/0/100/11/2              multimedia  ES1371/ES1373 / Creative Labs CT2518
/0/100/11/3              bus         USB2 EHCI Controller
/0/100/11/3/1    usb1    bus         EHCI Host Controller
/0/100/11/3/1/1          multimedia  VMware Virtual USB Video Device
/0/100/11/5              storage     SATA AHCI controller
```

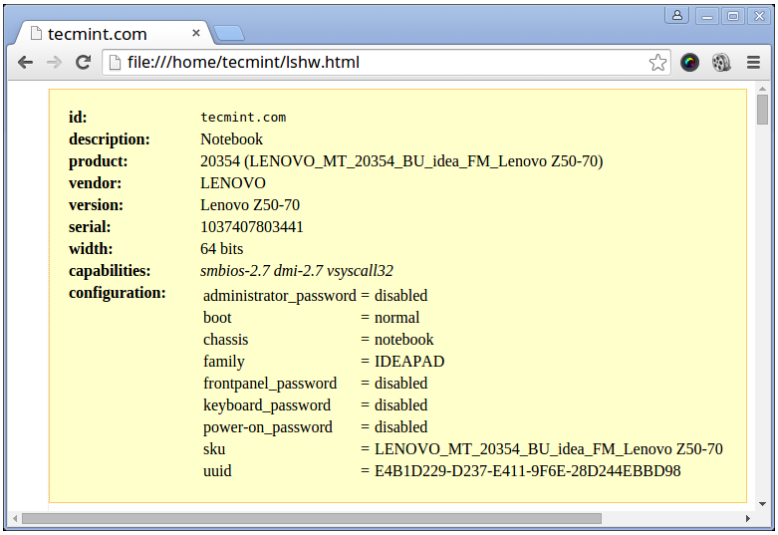
```
/0/100/15              bridge      PCI Express Root Port
/0/100/18.7            bridge      PCI Express Root Port

...
...

/0/1          scsi2     storage
/0/1/0.0.0      /dev/cdrom  disk        VMware SATA CD00
/0/1/0.0.0/0     /dev/cdrom  disk
/0/2          scsi3     storage
/0/2/0.0.0      /dev/dvd   disk        VMware SATA CD01
/1                   system
```

Según lo que se observa en la salida, es posible detectar rápidamente que se trata de una máquina virtual VMWare que además está conectada a la red mediante el adaptador ens33 y contiene varios volúmenes y un disco de 21GB montado, con lo que por ejemplo podremos navegarlos para seguir obteniendo información.

Así por ejemplo, si observamos el apartado de red en detalle, podemos observar que la máquina conectó por última vez a la red asociada al adaptador ens33 con la 172.16.60.162. Esto nos permitiría descubrir las posibles conexiones e interfaces de la máquina. Aunque es algo que podemos obtener de forma más detallada con **ifconfig**, esta sería una buena opción para una vista preliminar en un solo comando.

```
*-network
     description: Ethernet interface
     product: 82545EM Gigabit Ethernet Controller (Copper)
     vendor: Intel Corporation
     physical id: 1
     bus info: pci@0000:02:01.0
     logical name: ens33
     version: 01
     serial: 00:0c:29:f5:1f:7d
     size: 1Gbit/s
     capacity: 1Gbit/s
     width: 64 bits
     clock: 66MHz
     capabilities: pm pcix bus_master cap_list rom ethernet physical logical tp 10bt 10bt-fd 100bt 100bt-fd 1000bt-fd autonegotiation
     configuration: autonegotiation=on broadcast=yes driver=e1000 driverversion=7.3.21-k8-NAPI duplex=full ip=172.16.60.162 latency=0 link=y$
     resources: irq:19 memory:fd5c0000-fd5dffff memory:fdff0000-fdffffff ioport:2000(size=64) memory:fd500000-fd50ffff
```

Ya que la información es bastante densa y puede ser difícil de navegar, una opción interesante es lanzar el comando **sudo lshw -html > lshw.html** que volcará al fichero lshw.html la salida del comando en formato HTML que podremos explorar con cualquier navegador web.



También posible obtener la información detallada de cada una de estas secciones con los comandos "ls*" asociados:

- **lscpu** Muestra información sobre arquitectura, procesadores, familia del modelo CPU, caché...

- **lsblk** (Linux Block Device Information). Nos permite listar todos los volúmenes existentes en la máquina, su estado, punto de montaje (si lo está), tamaño y porcentaje de uso. Ejemplo:

```
NAME     MAJ:MIN RM   SIZE RO TYPE MOUNTPOINT
sda        8:0    0 931.5G  0 disk
├─sda1     8:1    0  1000M  0 part
├─sda2     8:2    0   260M  0 part /boot/efi
├─sda3     8:3    0  1000M  0 part
├─sda4     8:4    0   128M  0 part
├─sda5     8:5    0 557.1G  0 part
├─sda6     8:6    0    25G  0 part
├─sda7     8:7    0  14.7G  0 part
├─sda8     8:8    0     1M  0 part
├─sda9     8:9    0 324.5G  0 part /
└─sda10    8:10   0   7.9G  0 part [SWAP]
sdb        8:16   1         0 disk
```

En la imagen anterior podemos ver donde se encuentra la partición de arranque (/boot/efi) y que además la máquina contiene una partición de intercambio SWAP de 7.9GB de la que llegado el caso sería posible extraer información de la última ejecución de procesos y aplicaciones de la máquina, incluso si esta se detiene, al contrario que con la RAM, usando por ejemplo comandos como **hd (hexdump).** Mas info: https://www.geeksforgeeks.org/hexdump-command-in-linux-with-examples/

- **fdisk**. Muestra información relativa al Linux File System de la máquina. Un ejemplo de la salida de **sudo fdisk -l** sería la siguiente:

```
WARNING: GPT (GUID Partition Table) detected on '/dev/sda'! The util fdisk doesn


Disk /dev/sda: 1000.2 GB, 1000204886016 bytes
255 heads, 63 sectors/track, 121601 cylinders, total 1953525168 sectors
Units = sectors of 1 * 512 = 512 bytes
Sector size (logical/physical): 512 bytes / 4096 bytes
I/O size (minimum/optimal): 4096 bytes / 4096 bytes
Disk identifier: 0xcee8ad92

   Device Boot      Start         End      Blocks   Id  System
/dev/sda1               1  1953525167   976762583+  ee  GPT
Partition 1 does not start on physical sector boundary.
```

Cabe destacar que la mayoría de comandos anteriores hacen uso del directorio **/proc** que se detallará más adelante para obtener la información que posteriormente muestran de forma amigable al usuario.

1.2.3. Identificando el proveedor del servicio y datos relativos a la suscripción

En primer lugar, será necesario conocer si la máquina se trata de un servidor físico o virtualizado. Para ello, podemos varias herramientas, como por ejemplo **lshw** o **dmidecode** que puede instalarse en Ubuntu con el siguiente comando:

$ sudo apt-get install dmidecode

Podremos conocer si una máquina es física o virtual, además de la tecnología de virtualización empleada consultando la información relativa al ***Product*** y ***Vendor*** asociados.

Así, para obtener el fabricante, podremos ejecutar el comando **$ sudo dmidecode -s system-manufacturer** que nos proporcionará una salida similar a la siguiente:

```
admin1@mru8:~$ sudo dmidecode -s system-manufacturer
[sudo] password for admin1:
VMware, Inc.
```

Como se puede observar en la imagen anterior, en este caso la máquina se trata de una instancia virtualizada mediante VMWare.

Si se tratara de una máquina física obtendríamos una salida similar a la siguiente:

```
Dell Inc.
```

Mientras que para otros servicios de virtualización como VirtualBox obtendríamos la siguiente:

```
innotek GmbH
```

Y en el caso de KVM/QEMU, simplemente:

QEMU

También podríamos obtener esa información consultando la sección relativa al producto en la salida de **dmidecode** mediante el comando:

$ sudo dmidecode | grep Product

Que nos proporcionará una salida similar a la siguiente (en este caso, una máquina virtual en VMware):

```
admin1@mru8:~$ sudo dmidecode | grep Product
        Product Name: VMware Virtual Platform
        Product Name: 440BX Desktop Reference Platform
```

Algunos ejemplos para otros entornos:

[Sistema Físico Dell]

Inspiron N5050

[Máquina virtual en VirtualBox]

VirtualBox

[Máquina virtual en KVM/QEMU]

Standard PC (Q35 + ICH9, 2009)

Si ejecutamos esta misma operación en una máquina virtual Ubuntu de Amazon Web Services (AWS) EC2, obtendremos una salida similar a la siguiente:

```
ubuntu@i[redacted]:~$ sudo dmidecode | egrep -i 'manufacturer|product'
        Manufacturer: Xen
        Product Name: HVM domU
        Manufacturer: Xen
        Manufacturer: Intel
```

Mas opciones:

https://ostechnix.com/check-linux-system-physical-virtual-machine/

Una vez identificado el sistema como una máquina virtual, un ejercicio interesante es conocer cual es el proveedor cloud en el que se encuentra, lo que nos permitirá conocer mejor entorno de la máquina y si es posible, recabar y contrastar información mediante los paneles y consolas de administración que estos proveedores suelen ofrecer (grupos de seguridad, conectividad, accesos, discos, copias de seguridad, etc).

A continuación, se muestra cómo obtener el proveedor de servicio de una instancia para algunos de los principales servicios de virtualización en la nube. Aunque existe una gran mayoría de ellos y la información puede variar, es sencillo obtener información actualizada sobre cómo identificar cada uno de ellos.

AWS EC2:

Un posible método es obtener información sobre el UUID del Hypervisor. Si el fichero **/sys/hypervisor/uuid** existe, bastará con realizar un mostrado del mismo y comprobar si el UUID comienza por los caracteres “EC2”:

```
cat /sys/hypervisor/uuid
```

Lo que nos devolverá una salida similar a la siguiente:

```
ec2e1916-9099-7caf-fd21-012345abcdef
```

Para instancias HVM es posible obtenerlo usando **dmidecode**:

```
sudo dmidecode --string system-uuid
```

En ambos casos, existe una pequeña posibilidad de que instancias no pertenecientes a AWS contengan UUID que comiencen con estos caracteres, debido al proceso que implementen para la generación de UUID.

Mas información: https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/identify_ec2_instances.html

Azure:

Un método sencillo y bastante fiable es comprobar la existencia de la opción **"unknown-245"**, una opción propietaria utilizada únicamente por instancias de Azure y que se usa para la obtención de IPs a través de un servidor Azure DHCP. Así, podríamos ejecutar el siguiente script para comprobar si una instancia Ubuntu pertenece a Azure o no.

```
if `grep -q unknown-245 /var/lib/dhcp/dhclient.eth0.leases`; then
    echo "Running in an Azure VM"
fi
```

Otra aproximación consiste en leer la configuración del propio Microsoft Azure Linux Agent (waagent), que siempre vendrá instalado en las instancias virtualizadas tipo Linux. Así, intentaremos hacer una conexión local a dicho agente y si obtenemos una respuesta

distinta a un timeout, podremos afirmar que se trata de una instancia Azure. Podemos realizar esta comprobación mediante el siguiente script:

```
curl –connect-timeout 1 `grep FullConfig /var/lib/waagent/GoalState.1.xml | perl -pe 's/<.?FullConfig>//g; s/\s//g'` && echo azure || echo no-azure
```

Por último, siempre será posible buscar la existencia de logs emitidos por el propio Microsoft Azure Linux Agent, aunque ya que es posible instalarlo en cualquier instancia mediante diferentes gestores de paquetes, es recomendable buscar en estos un rastro reciente de comunicación con el proveedor del servicio. Podemos hacerlo mediante el siguiente comando:

```
cat /var/log/waagent.log | grep -i "Retrieved GoalState from Windows Azure Fabric"
```

Caso general

Para la mayoría de proveedores se pueden utilizar las dos siguientes opciones:

- **Consultar el mensaje del día:** Por lo general los proveedores suelen incluir un mensaje del día personalizado, conocido por sus siglas en ingles MOTD (Message Of The Day) que nos permitirá deducir cual es el proveedor. Obtendremos en todo caso este mensaje automáticamente cada vez que hagamos login en la máquina. En el caso de Ubuntu a partir de la versión 16 podremos obtenerlo consultando el fichero**:** **/var/run/motd.dynamic** y **/etc/motd** para las versiones anteriores.
- **Obtener la IP pública de la instancia** y comprobar a qué proveedor pertenece dicho rango de IPs. Una opción sencilla es aplicar un **whois** a dicha IP.

Ejercicio:

Diseña un script que permita conocer si la instancia en la que se lanza es física o virtual y en el segundo caso, que tecnología de virtualización utiliza y cual es su proveedor.

1.3. RECOLECCIÓN DE LOGS

A diferencia de las métricas, las cuales tienen una representación numérica, los logs son datos que se recogen en formato texto y son producidos por el funcionamiento de las aplicaciones.

Durante el funcionamiento de una aplicación, esta va realizando acciones siguiendo un determinado algoritmo que soluciona una de serie de problemas para los cuales ha sido diseñada. En este proceso, el programador suele incluir mensajes que van aportando información sobre el correcto o incorrecto funcionamiento de la misma. Esta información se conoce por logs de funcionamiento y es una información muy valiosa para conocer que está ocurriendo en nuestros sistemas.

El tratamiento de los logs no siempre es tarea sencilla, pudiendo convertirse en una pesadilla si no se cumplen con las buenas prácticas desde el propio programador de la aplicación hasta el personal de soporte que se encarga de la administración, recolección y seguimiento de los logs.

Para realizar un correcto tratamiento de los logs, es fundamental darle la importancia que merecen y seguir algunas consideraciones.

- **Información mínimamente necesaria pero suficiente.** Esto es imprescindible, no hay nada peor que, tras haber ocurrido un incidente, ir a buscar el archivo de logs y encontrarse con medio giga de información acumulada de semanas de funcionamiento, donde la mayoría de información no aporta nada y tener que emplear horas y horas buscando y analizando para poder encontrar lo que se andaba buscando...
  En la mayoría de las ocasiones, basta con indicar donde se produjo el error (paquete, clase, método, línea, ...) y una breve (pero muy breve) descripción del problema y en caso de que sea posible, información sobre algún identificador que permita rastrear con que datos concretamente ha provocado el error.
  Por ejemplo, imaginemos que ha habido un error al iniciar sesión un determinado usuario.
  Una entrada del log podría contener la fecha y hora, la línea, paquete, módulo, método, etc., el mensaje "sesión invalida", y el identificador del usuario que provocó el error.

- **¡Formato estrictamente! unificado de los logs.** Este es uno de los mayores problemas, sobre todo en sistemas que por su naturaleza producen muchos logs, o cuando se disponen de muchas instancias. En escenarios como los anteriores, se hace inmanejable tratar los logs manualmente, por esto, se recurre a herramientas que permiten tratar los logs y facilitarnos la tarea de búsqueda e interpretación de los mismos. Para poder integrar este tipo de herramientas, es obligatorio que todos los logs cumplan un formato (¡¡y si puede ser estándar mejor!!) ya que de otra forma se hace imposible el parseo de los logs.

Campus Internacional CIBERSEGURIDAD
ENIIT INNOVA IT BUSINESS SCHOOL
UCAM UNIVERSIDAD CATÓLICA DE MURCIA

- **Protege tus logs!!.** Al igual que los logs son un recurso muy valioso para nosotros, también lo son para un atacante. En un primer lugar, aportan retroalimentación al atacante, que puede usarlos para comprobar si el ataque está funcionando o no. También le permiten identificar recursos del sistema, ejemplos de peticiones, información sobre los ciclos de funcionamiento, temporización de los mismos, usuarios y cualquier otra información que nuestras aplicaciones almacenen en los logs.
- **Rota tus logs correctamente!!**. Como es de esperar, si estamos escribiendo en disco logs infinitamente el disco se acabará llenando, siendo imprescindible ir desechando los antiguos. Existen numerosas herramientas que facilitan esa tarea. Es importante ajustar correctamente la persistencia de nuestros logs para garantizar que siempre tenemos disponible la cantidad de logs suficiente que vamos a necesitar. También es frecuente que los logs sean enviados a un servidor de logs centralizado con la capacidad de almacenamiento y tratamiento de los mismos, liberando así de esta labor a la máquina donde se ejecuta la aplicación.

Enlace interesante…

https://www.eugdpr.org/

Unificado de logs y monitorización de instancias múltiples

Cuando disponemos de varias máquinas que debemos gestionar, especialmente si componen un mismo servicio, como por ejemplo una aplicación web de cierto tamaño con instancias replicadas para conseguir alta disponibilidad.

Algunos ficheros de logs y comandos típicos

- **lastlog:** Muestra la información del último login de los usuarios del sistema.
- **last -f /var/log/wtmp:** Muestra la actividad completa de login y logout de los usuarios.
- **/var/log:** Directorio donde se recogen la mayoría de los logs del sistema.
- **/var/log/message:** Mensajes de propósito general y cosas relacionadas con el sistema.
- **/var/log/auth.log:** Logs de autenticación. Tiempo, usuario y método.
- **/var/log/kern.log:** Logs del kernel.
- **/var/log/cron.log:** Logs de cron.
- **/var/log/maillog:** Logs de servicios de email.
- **/var/log/apache2:** Logs de apache.

- **/var/log/boot.log:** Logs de arranque del sistema.
- **/var/log/secure:** Logs de autenticación.
- **/var/log/utmp or /var/log/wtmp:** Fichero con el registro de login de usuarios.
- **/var/log/aptitude.log:** Logs de aptitude.
- **/var/log/faillog:** Intentos fallidos de login.
- **tail –f ficherodelog:** Muestra en tiempo real el flujo de escritura sobre el fichero indicado.

2. EL DIRECTORIO /PROC

El directorio **/proc** es un elemento peculiar de los sistemas Linux. Representa un sistema de ficheros virtual que no existe de forma permanente en el sistema operativo, ya que es creado junto al propio directorio cada vez que la máquina es arrancada. Además, sus archivos (de tamaño 0) no son ni binarios ni texto, sin embargo, pueden ser examinados y su contenido, desplegado.

Pese a todas las peculiaridades comentadas anteriormente, este directorio especial contiene una importante cantidad de detalles fundamentales del sistema, incluyendo el kernel, procesos y parámetros de configuración. De hecho, como se verá a continuación, estudiando el directorio /proc es posible conocer como trabajan la mayoría de comandos de Linux además de ejecutar algunas tareas administrativas.

En Linux, prácticamente todo es administrado como un archivo. Como se vio en la unidad 8.1, incluso los dispositivos son accedidos como archivos bajo el directorio /dev. Sin embargo, en el caso del directorio /proc, los archivos que contiene tampoco pueden considerarse “normales”, ya que se tratan de archivos virtuales, que no existen en el disco, pero que pueden ser listados y que el sistema operativo creará al vuelo cuando se solicite su lectura. Estos ficheros, que están siendo actualizados constantemente, incluyen una marca de tiempo actualizada, que permite conocer la antigüedad de la información que proveen.

Como se comentó en la unidad 8.1, la mayor parte de la información que contiene /proc está relacionada con la ejecución de procesos, por lo que algunos de estos fiheros son propiedad del fichero que los ejecutó y la mayoría de ellos tienen permisos de solo lectura. Además, dada la

Campus Internacional CIBERSEGURIDAD
ENIIT INNOVA IT BUSINESS SCHOOL
UCAM UNIVERSIDAD CATÓLICA DE MURCIA

importancia de la información que contiene, para examinar completamente el directorio /proc será necesario tener acceso root.

2.1. ORGANIZACIÓN Y ESTRUCTURA DEL DIRECTORIO /PROC

Los ficheros contenidos por /proc se distribuyen dentro de directorios virtuales y subdirectorios que los agrupan por temática o áreas y funcionalidades del sistema operativo.

Así, si ejecutamos como root el comando **ls /proc** obtendremos una salida equivalente a la siguiente:

```
#> ls /proc
1    2432  3340  3715  3762  5441  815       devices     modules
129  2474  3358  3716  3764  5445  acpi      diskstats   mounts
1290 248   3413  3717  3812  5459  asound    dma         mtrr
133  2486  3435  3718  3813  5479  bus       execdomains partitions
1420 2489  3439  3728  3814  557   dri       fb          self
165  276   3450  3731  39    5842  driver    filesystems slabinfo
166  280   36    3733  3973  5854  fs        interrupts  splash
2    2812  3602  3734  4     6     ide       iomem       stat
2267 3     3603  3735  40    6381  irq       ioports     swaps
2268 326   3614  3737  4083  6558  net       kallsyms    sysrq-trigger
2282 327   3696  3739  4868  6561  scsi      kcore       timer_list
2285 3284  3697  3742  4873  6961  sys       keys        timer_stats
2295 329   3700  3744  4878  7206  sysvipc   key-users   uptime
2335 3295  3701  3745  5     7207  tty       kmsg        version
2400 330   3706  3747  5109  7222  buddyinfo loadavg     vmcore
2401 3318  3709  3749  5112  7225  cmdline   locks       vmstat
2427 3329  3710  3751  541   7244  config.gz meminfo     zoneinfo
2428 3336  3714  3753  5440  752   cpuinfo   misc
```

Los directorios con un número por nombre corresponden con cada uno de los procesos en ejecución. Un autoenlace simbólico, apunta al proceso actual.

Entre el resto de archivos virtuales, algunos proveen información sobre el hardware, como por ejemplo **/proc/cpuinfo** o **/proc/interrupts.** Otros proporcionan información relacionada a los archivos, tales como **/proc/filesystems** o **/proc/partitions**. Los archivos situados bajo **/proc/sys** son relacionados a la configuración de parámetros del kernel.

Gracias a la información que proveen los ficheros de /proc, numerosas herramientas y programas (incluyendo a los del sistema operativo) consumen estos ficheros para obtener e interpretar la información que contienen. Por ejemplo, los comandos **top** o **free**, mostrarán información procedente de el fichero **/proc/meminfo** de forma actualizada y en un formato más afable a los usuarios. Si por ejemplo necesitamos conocer la información del kernel que está ejecutando la máquina podremos usar el comando **uname -srv** o bien ir directamente al fichero origen utilizando **cat /proc/versión**

A continuación de describen algunos de los ficheros virtuales más relevantes de /proc:

2.2. DIRECTORIOS VIRTUALES Y FICHEROS RELACIONADOS CON EL HARDWARE

- **/proc/meminfo**. Si ejecutamos **cat /proc/meminfo** obtendremos por consola algo similar a lo siguiente:

```
# cat /proc/meminfo
MemTotal:      483488 kB
MemFree:         9348 kB
Buffers:        6796 kB
Cached:       168292 kB
...
```

También hay varios archivos específicos relacionados a la RAM además de */proc/meminfo*, como por ejemplo ***/proc/iomem***, que muestra cuanta memoria RAM ha sido utilizada por el sistema y como se asigna a cada dispositivo y ***/proc/kcore***, que representa la memoría física de la RAM. A diferencia de otros archivos virtuales, ***/proc/kcore*** muestra un tamaño que es igual a la memoria RAM más una pequeña sobrecarga. (No tiene sentido realizar ***cat*** a este archivo, porque su contenido es binario y no se podría interpretar a simple vista).

- **/proc/iomem:** Este archivo muestra el mapa actual de la memoria del sistema para los diversos dispositivos. Un ejemplo de su contenido sería el siguiente:

```
00000000-0009fbff : System RAM
0009fc00-0009ffff : reserved
000a0000-000bffff : Video RAM area
000c0000-000c7fff : Video ROM
000f0000-000fffff : System ROM
00100000-07ffffff : System RAM
  00100000-00291ba8 : Kernel code
  00291ba9-002e09cb : Kernel data
e0000000-e3ffffff : VIA Technologies, Inc. VT82C597 [Apollo VP3]
e4000000-e7ffffff : PCI Bus #01
  e4000000-e4003fff : Matrox Graphics, Inc. MGA G200 AGP
  e5000000-e57fffff : Matrox Graphics, Inc. MGA G200 AGP
e8000000-e8ffffff : PCI Bus #01
  e8000000-e8ffffff : Matrox Graphics, Inc. MGA G200 AGP
ea000000-ea00007f : Digital Equipment Corporation DECchip 21140 [FasterNet]
  ea000000-ea00007f : tulip
ffff0000-ffffffff : reserved
```

La primera columna muestra los registros de memoria utilizados por cada uno de los diferentes tipos de memoria. La segunda columna indica el tipo de memoria de dichos registros y muestra qué registros de memoria son usados por el kernel dentro de la RAM

del sistema o, si la tarjeta NIC tiene múltiples puertos Ethernet, los registros de memoria asignados para cada puerto.

- **/proc/buddyinfo:** Este archivo se utiliza principalmente para diagnosticar problemas de fragmentación de memoria. Utilizando el algoritmo buddy, cada columna representa el número de páginas de un cierto orden (de un cierto tamaño) que están disponibles en un momento dado. Por ejemplo, para la zona DMA (acceso directo a memoria) de la máquina mostrada en la salida obtenida más adelante para este fichero, hay 90 de 2^(2*PAGE_SIZE) pedazos de memoria. De forma similar, hay 6 de 2^(2*PAGE_SIZE) pedazos, y 2 de 2^(2*PAGE_SIZE) pedazos de memoria disponibles.

  La fila **DMA** hace referencia a los primeros 16 MB en un sistema, la fila **HighMem** referencia toda la memoria mayor que 4 GB en un sistema, y la fila **Normal** se refiere a toda la memoria en medio de las anteriores.

Lo siguiente es un ejemplo de la salida típica de **/proc/buddyinfo**:

```
Node 0, zone      DMA     90     6     2     1     1     ...
Node 0, zone   Normal   1650   310     5     0     0     ...
Node 0, zone  HighMem      2     0     0     1     1     ...
```

- ***/proc/apm*:** Provee información sobre "Advanced Power Management", si es que esta instalado. Es usado por el comando **apm**.
- ***/proc/acpi***: Un directorio similar al anterior que ofrece bastantes datos sobre el más moderno ACPI (Advanced Configuration and Power Interface). Por ejemplo, para ver el equipo está conectada a la corriente AC, es posible ejecutar el comando **cat /proc/acpi/ac_adapter/AC/state** o en algunas distros **cat /proc/acpi/ac_adapter/ACAD/state**, con lo que se obtendrá **"on line"** u **"off line"** dependiendo de si la máquina se encuentra conectada a la corriente o no.

- ***/proc/cpuinfo***: Muestra datos sobre el procesador del equipo. Por ejemplo, **cat /proc/cpuinfo** mostraría un listado similar al siguiente:

```
processor      : 0
vendor_id      : AuthenticAMD
cpu family     : 6
model          : 8
model name     : Mobile AMD Athlon(tm) XP 2200+
stepping       : 1
cpu MHz        : 927.549
cache size     : 256 KB
...
```

Esta salida muestra que solo se dispone de un procesador, marcado como 0 de la familia 80686 (El '6' en la familia de cpu es dígito medio): un AMD Athlon XP, corriendo a poco menos de 1GHZ.

- ***/proc/devices***: Despliega todos los dispositivos actuales configurados y cargados de caracter y bloque. ***/proc/ide*** y ***/proc/scsi*** proveen datos sobre dispositivos IDE y SCSI.
- ***/proc/ioports***: Muestra información sobre las regiones usadas para comunicaciones de E/S (I/O) de los dispositivos listados en el fichero anterior.
  Un posible listado del fichero sería el siguiente:

```
0000-001f : dma1
0020-003f : pic1
0040-005f : timer
0060-006f : keyboard
0070-007f : rtc
0080-008f : dma page reg
00a0-00bf : pic2
00c0-00df : dma2
00f0-00ff : fpu
0170-0177 : ide1
01f0-01f7 : ide0
02f8-02ff : serial(auto)
```

```
0376-0376 : ide1
```

```
03c0-03df : vga+
03f6-03f6 : ide0
03f8-03ff : serial(auto)
0cf8-0cff : PCI conf1
d000-dfff : PCI Bus #01
e000-e00f : VIA Technologies, Inc. Bus Master IDE
  e000-e007 : ide0
  e008-e00f : ide1
e800-e87f : Digital Equipment Corporation DECchip 21140 [FasterNet]
  e800-e87f : tulip
```

La primera columna indica el rango de direcciones de los puertos de entrada y salida reservado para el dispositivo listado en la segunda columna.

- ***/proc/dma***: Muestra los canales "Direct Memory Access" en uso. Recordemos que DMA permite a los dispositivos acceder directamente a la memoria principal sin necesidad de pasar por la CPU.

Finalmente, hay muchos archivos y directorios relacionados con el Hardware, tal como */proc/interrupts* y */proc/irq*, */proc/pci* (para todos los dispositivos PCI), */proc/bus* y demás, pero todos estos incluyen información muy específica que la mayoría de los usuarios no necesitarán.

2.3. DIRECTORIOS Y FICHEROS VIRTUALES QUE OFRECEN INFORMACIÓN DEL SISTEMA

- ***/proc/loadavg***: Un archivo relacionado al anterior que muestra la carga promedio del procesador; su información incluye el uso de CPU en el último minuto, últimos cinco minutos y últimos 15 minutos, así como el número de procesos actuales ejecutándose.
- ***/proc/cmdline***: Muestra los parámetros que fueron pasados al kernel al momento del arranque (boot time). Así por ejemplo, la salida: ***ro root=/dev/disk/by-id/scsi-SATA_FUJITSU_MHS2040_NLA5T3314DW3-part3 vga=0x317 resume=/dev/sda2 splash=silent***, indica que el kernel está montado en modo sólo lectura (**ro**) y en qué partición se encuentra la raíz del sistema de ficheros (en este caso ***/dev/disk/by-id/scsi-SATA_FUJITSU_MHS2040_NLA5T3314DW3-part3)***. Además, muestra qué modo VGA se está usando (***vga=0x317***), qué dispositivo de swap se utilizará en el encendido tras hibernación (***resume=/dev/sda2)*** y que al inicio no se mostrará la vista de splash (***splash=silent).*** Se

mostrará por tanto una lista de mensajes de inicio que también podría ser suprimida con la opción "**quiet**". En la web https://www.kernel.org/doc/Documentation/admin-guide/kernel-parameters.txt puede encontrarse información detallada sobre todos los parámetros posibles del kernel.

/proc/modules: Este archivo muestra una lista de todos los módulos cargados en el sistema. Su contenido variará dependiendo de la configuración y uso de cada sistema, pero debería organizarse de forma similar al siguiente ejemplo de salida:

```
nfs      170109  0 -        Live 0x129b0000
lockd    51593   1 nfs,     Live 0x128b0000
nls_utf8 1729    0 -        Live 0x12830000
vfat     12097   0 -        Live 0x12823000
fat      38881   1 vfat,    Live 0x1287b000
autofs4  20293   2 -        Live 0x1284f000
sunrpc   140453  3 nfs,lockd, Live 0x12954000
3c59x    33257   0 -        Live 0x12871000
uhci_hcd 28377   0 -        Live 0x12869000
md5      3777    1 -        Live 0x1282c000
ipv6     211845 16 -        Live 0x128de000
ext3     92585   2 -        Live 0x12886000
jbd      65625   1 ext3,    Live 0x12857000
dm_mod   46677   3 -        Live 0x12833000
```

- La primera columna contiene el nombre del módulo.
- La segunda columna se refiere al tamaño de la memoria del módulo, en bytes.
- La tercera columna lista cuántas instancias del módulo están cargadas actualmente. Un valor de cero representa un módulo sin cargar.
- La cuarta columna indica si el módulo depende de que otro módulo esté presente para poder funcionar, y lista esos otros módulos.
- La quinta columna lista en qué estado de carga se encuentra el módulo: **Live**, **Loading** o **Unloading** son los únicos valores posibles.

- La sexta columna lista el desplazamiento de memoria del kernel actual para el módulo cargado. Esta información puede ser útil para propósitos de depuración o para herramientas de perfiles, tales como **oprofile**.

Información muy similar a la de este fichero puede obtenerse mediante el uso del comando **/sbin/lsmod**

- ***/proc/stat***: También proporciona estadísticas, pero tomando en cuenta el último boot (arranque).
- ***/proc/uptime***: Un archivo pequeño que solo tiene dos números: por cuantos segundos tu sistema ha estado encendido, y cuantos segundos ha estado inactivo.
- ***/proc/filesystems***: Muestra cuáles tipos de sistemas de archivos (filesystems) están soportados por tu kernel. Una porción de esto puede lucir como esto:

```
nodev   sysfs
nodev   rootfs
nodev   bdev
nodev   proc
nodev   cpuset
...some lines snipped...
nodev   ramfs
nodev   hugetlbfs
nodev   mqueue
        ext3
nodev   usbfs
        ext2
nodev   autofs
```

La primera columna muestra si el sistema de archivos está montado en un dispositivo de bloques. En este caso, existen particiones configuradas con ***ext2*** y ***ext3***.

- ***/proc/mounts***: Muestra todos los "mounts" o puntos de montaje actuales para la máquina (la salida es muy similar a ***/etc/mtab***). De forma similar, los ficheros, ***/proc/partititons*** y ***/proc/swaps*** muestran todas las particiones y el espacio swap respectivamente.

- ***/proc/fs***: Si estás exportando sistemas de archivos con NFS, este directorio tiene entre sus muchos subdirectorios y archivos a */proc/fs/nfsd/exports*, que muestra los sistemas de archivos que están siendo compartidos y sus permisos.
- ***/proc/net***: A través de este directorio puede obtenerse una gran cantidad de información relacionada con el ámbito de red. Describir cada archivo en este directorio requeriría demasiado espacio, pero es importante destacar incluye *dev* (al menos todos los dispositivos de red), varios archivos relacionados al firewall **iptables**, estadísticas de red y sockets, información sobre wireless y más.

Más directorios y ficheros de /proc e información detallada sobre los mismos: https://web.mit.edu/rhel-doc/4/RH-DOCS/rhel-rg-es-4/s1-proc-topfiles.html

2.4. REPRESENTACIÓN DE PROCESOS EN /PROC

Como se ha comentado en la sección anterior, los procesos son representados en /proc mediante directorios nombrados con el identificador de proceso al que pertenecen. Una vez que un proceso termina, su directorio asociado en /proc desaparece automáticamente. Al examinar cualquiera de los directorios asociados a un proceso, se obtendrá una salida similar a la siguiente:

```
attr          cpuset   fdinfo    mountstats  stat
auxv          cwd      loginuid  oom_adj     statm
clear_refs    environ  maps      oom_score   status
cmdline       exe      mem       root        task
coredump_filter  fd    mounts    smaps       wchan
```

Veamos más de cerca a los archivos principales:

- ***cmdline***: Contiene el comando que inicio el proceso, con todos sus parámetros.
- ***cwd***: Enlace simbólico al directorio actual de trabajo (current working directory cwd) del proceso; enlaces exe a los procesos ejecutables y enlaces root a su directorio superior.
- ***environ***: Muestras todas las variables de entorno para el proceso.
- ***fd***: Contiene los descriptores de archivo para el proceso, mostrando que archivos o dispositivos lo están usando.
- ***maps, statm and mem***: Trabaja con la memoria en uso por el proceso.

- ***stat and status***: Provee información acerca del estatus del proceso, pero el segundo es mucho más claro que el primero.
- ***limits***: muestra de una forma estructurada la lista de limites aplicados a este proceso junto con el límite concreto en unidades, así como la unidad en la que se miden. Listar el fichero limits es una buena forma de conocer si existe algún límite que pueda estar afectando a la ejecución normal de un proceso en combinación con otras herramientas que ofrezcan información sobre el consumo de recursos que generan. Un ejemplo de salida al listar el este fichero sería el siguiente:

```
admin1@mru8:/proc/1899$ cat limits
Limit                     Soft Limit           Hard Limit           Units
Max cpu time              unlimited            unlimited            seconds
Max file size             unlimited            unlimited            bytes
Max data size             unlimited            unlimited            bytes
Max stack size            8388608              unlimited            bytes
Max core file size        0                    unlimited            bytes
Max resident set          unlimited            unlimited            bytes
Max processes             3603                 3603                 processes
Max open files            1024                 1048576              files
Max locked memory         16777216             16777216             bytes
Max address space         unlimited            unlimited            bytes
Max file locks            unlimited            unlimited            locks
Max pending signals       3603                 3603                 signals
Max msgqueue size         819200               819200               bytes
Max nice priority         0                    0
Max realtime priority     0                    0
Max realtime timeout      unlimited            unlimited            us
```

Estos archivos plantean varios retos de programación de shell scripts. Por ejemplo, si se desea cazar a los procesos zombie, es posible buscar en todos los directorios numerados y verificar si aparece "(Z) Zombie" en el archivo *status*. Una vez necesitaba verificar si cierto programa se estaba ejecutando: Lo que se puede hacer en este caso, es escanear y mirar a los archivos *cmdline*, buscando por la cadena indicada. (También se puede hacer esto con la salida del programa ps, pero esta es la opción “mas directa”). Y queremos programar un programa que se vea mejor que top, toda la información necesaria estaría aqui.

Ejercicio:

Diseñar un programa que liste en tiempo real los procesos del sistema que están en estado "Zombie" a través de la información disponible en /proc

2.5. FICHEROS DE ESCRITURA EN /PROC. CONFIGURANDO EL SISTEMA

A pesar de que la mayoría de los ficheros de /proc son de solo lectura, existen algunos que permiten ser escritos, que permiten realizar cambios en los parámetros del kernel. Se trata por tanto de ficheros especialmente delicados cuya modificación inadecuada puede dañar el sistema, por lo que deben ser accedidos con cuidado.

Para determinar qué ficheros pueden ser configurados o son de solo lectura, podemos usar el comando **ls -ld**. Así por ejemplo, si aplicamos el comando **ls -ld /proc/kernel/*** obtendremos una salida similar a la siguiente:

```
dr-xr-xr-x 0 root root 0 2008-01-26 00:49 pty
dr-xr-xr-x 0 root root 0 2008-01-26 00:49 random
-rw-r--r-- 1 root root 0 2008-01-26 00:49 acct
-rw-r--r-- 1 root root 0 2008-01-26 00:49 acpi_video_flags
-rw-r--r-- 1 root root 0 2008-01-26 00:49 audit_argv_kb
-r--r--r-- 1 root root 0 2008-01-26 00:49 bootloader_type
-rw------- 1 root root 0 2008-01-26 00:49 cad_pid
-rw------- 1 root root 0 2008-01-26 00:49 cap-bound
```

Los archivos que contienen el atributo "w" (permisos de escritura) podrán ser modificados, lo que significan que podremos modificar el kernel de algún modo modificándolos. En cambio, ficheros como **bootloader_type**, no podrán ser modificados, ya que solo tiene permisos de lectura.

Campus Internacional CIBERSEGURIDAD
ENIIT INNOVA IT BUSINESS SCHOOL
UCAM UNIVERSIDAD CATÓLICA DE MURCIA

2.5.1. /proc/sys

Para cambiar un archivo, se puede usar algo como esto:

```
echo 10 > /proc/sys/vm/swappiness
```

Este ejemplo en particular cambiará el rendimiento de la paginación de la memoria virtual (lo usaremos en la siguiente unidad). Estos cambios son solo temporales, y sus efectos desaparecerán una vez reiniciado el sistema; Sería necesario usar **sysctl** o el archivo de configuración ***/etc/sysctl.conf*** para aplicar los cambios de manera permanente.

Veamos el nivel superior del directorio ***/proc/sys***:

debug: Contiene información para depuración (debugging). Esto es muy útil por ejemplo si se trabaja en el desarrollo del kernel, módulos o drivers.

dev: Proveé parámetros para dispositivos específicos del sistema; por ejemplo, revisa el directorio ***dev/cdrom***.

fs: Ofrece datos sobre cada posible aspecto del sistema de archivos.

kernel: Permite afectar la configuración del kernel y su operación directamente.

net: Te permite controlar aspectos relacionados a la red. Requiere ser cuidadoso, porque puede conllevar la perdida de conectividad en la máquina.

vm: Trata con el subsistema VM.

Ejercicio:

Un requisito indispensable si se pretende analizar una máquina de la que se sospecha que pueda estar comprometida (e incluso si solo se quiere analizar su funcionamiento), es conocer exactamente qué procesos hay ejecutándose en la misma, qué métricas podemos llegar a extraer de cada uno, sus significados y valores propios de un funcionamiento normal.

Aunque en este apartado se ha explicado la cantidad de información que aporta el directorio /proc y que éste suele ser la fuente en la que se basan la mayoría de las herramientas de visualización de información del sistema, en un entorno real, utilizarlo directamente es poco práctico. Para ello, podemos utilizar herramientas de monitorización como top o htop, que agutinan la mayor parte de la información relativa a ejecución de procesos en una vista bastante simplificada, pero suficientemente

Campus Internacional CIBERSEGURIDAD
ENIIT INNOVA IT BUSINESS SCHOOL
UCAM UNIVERSIDAD CATÓLICA DE MURCIA

configurable. Aunque las hay mucho más avanzadas, suelen ser mas pesadas y complejas de instalar. Por ejemplo htop se puede encontrar en cualquier gestor de paquetes e incluso se encuentra ya incluida en multitud de distribuciones.

Elabora un informe con las principales funcionalidades de htop así como una serie de ejemplos de uso de las que estimes más interesantes en la búsqueda de anomalías en una máquina.

3. ¿QUÉ EXPONE LA MÁQUINA A INTERNET?

Uno de los detalles más importantes que nos interesa conocer de una máquina a la que acabamos de conseguir acceso (especialmente en procesos de análisis, pentesting, auditoría o hardening) es qué esta exponiendo dicha máquina a internet. Esto nos permitirá por ejemplo detectar procesos que tengan abiertas conexiones no deseadas, como podría ser una shell inversa o cualquier backdoor. Para ello, además de consultar las diferentes reglas de acceso y firewalls que pueda tener configuradas (como se explicó en la unidad 8.1), podemos valernos de diferentes herramientas específicas.

Nmap:

Nmap permite:

- Descubrimiento de hosts
- Descubrimiento y enumeración de puertos
- Descubrimiento de servicios y detección de versiones
- Detección de sistema operativo
- Detección de Hardware (MAC) address
- Detección de vulnerabilidad y de posibilidad de ejecución de exploit asociados usando los Nmap scripts (NSE)

Campus Internacional CIBERSEGURIDAD
ENIIT INNOVA IT BUSINESS SCHOOL
UCAM UNIVERSIDAD CATÓLICA DE MURCIA

```
notwist@notwist:~$ nmap localhost

Starting Nmap 4.20 ( http://insecure.org ) at 2007-04-02 15:50 CEST
Interesting ports on localhost (127.0.0.1):
Not shown: 1691 closed ports
PORT     STATE SERVICE
22/tcp   open  ssh
80/tcp   open  http
139/tcp  open  netbios-ssn
445/tcp  open  microsoft-ds
631/tcp  open  ipp
3306/tcp open  mysql

Nmap finished: 1 IP address (1 host up) scanned in 0.213 seconds
notwist@notwist:~$
```

Ejemplo de salida de una ejecución básica de Nmap en local

Podemos utilizar nmap para obtener numerosa información al respecto: https://null-byte.wonderhowto.com/how-to/use-nmap-7-discover-vulnerabilities-launch-dos-attacks-and-more-0168788/

Algunos comandos interesantes. Las plantillas de temporizado (-T) controlan el tiempo de los escaneos, siendo los números más altos los más rápidos:

COMMANDO	DESCRIPCIÓN
nmap -sP 10.0.0.0/24	Escanea la red utilizando ping listando las máquinas que respondan.
nmap -p 1-65535 -sV -sS -T4 target	Ejecuta un escaneo complete de puertos TCP con detección de servicios y versions asociados. Por lo general es el primer escaneo que suele lanzar un atacante
nmap -v -sS -A -T4 target	Ejecuta un escaneo SYN (-sS) sigiloso con timing T4 incluyendo detección de sistema operativo y version utilizando traceroute y scripts contra los servicios detectados. Muestra la salida en modo verboso
nmap -v -sS -A -T5 target	Igual al anterior, pero con timing T5

COMMANDO	DESCRIPCIÓN
nmap -v -sV -O -sS -T5 target	Imprime de forma verbose el resultado de un escaneo SYN sigiloso con detección de sistema operativo y versión
nmap -v -p 1-65535 -sV -O -sS -T4 target	Igual al anterior pero con timing T4 e incluyendo un escaneo completo de puertos.
nmap -v -p 1-65535 -sV -O -sS -T5 target	Igual que el anterior pero con timing T5

Más ejemplos de comandos útiles: https://highon.coffee/blog/nmap-cheat-sheet/

Mencionar los scripts de nmap y sintetizar este post: https://www.welivesecurity.com/la-es/2015/02/12/auditando-nmap-scripts-escanear-vulnerabilidades/

Lsof (List Open Files):

Se trata de una herramienta incluida por defecto en la gran mayoría de distribuciones UNIX/Linux que permite mostrar todos los ficheros que tienen abiertos los procesos del sistema, lo que entre otros incluye sockets y pipes de red. Gracias a la importante cantidad de parámetros y configuraciones que permita, podemos obtener la siguiente información por cada proceso:

- Nombre e id del proceso,
- Nombre o id del usuario que lanzó dicho proceso,
- El descriptor de fichero junto con el modo de apertura (lectura,escritura,etc..),
- El tipo de nodo asociado al archivo: Tubería (FIFO), Socket IPV4, Socket IPv6, Directorio, Bloque de Dispositivo, etc..
- La ruta absoluta del archivo o bien información relativa según el tipo,
- El contexto de seguridad SELinux

Lsoft es una de las mejores maneras de asociar procesos a puertos y conexiones abiertas en la máquina. La siguiente imagen muestra los distintos procesos de la máquina que tienen puertos TCP en escucha

```
ubuntu@i[redacted]:~$ sudo lsof -i -sTCP:LISTEN
COMMAND       PID            USER   FD   TYPE     DEVICE SIZE/OFF NODE NAME
systemd-r     463 systemd-resolve   13u  IPv4      16975      0t0  TCP localhost:domain (LISTEN)
sshd          692            root    3u  IPv4      19724      0t0  TCP *:ssh (LISTEN)
sshd          692            root    4u  IPv6      19726      0t0  TCP *:ssh (LISTEN)
docker-pr 3574324            root    4u  IPv6 1001029064      0t0  TCP *:27017 (LISTEN)
docker-pr 3574336            root    4u  IPv6 1001029116      0t0  TCP *:15672 (LISTEN)
docker-pr 3574363            root    4u  IPv6 1001029163      0t0  TCP *:amqp (LISTEN)
docker-pr 3574585            root    4u  IPv6 1001029588      0t0  TCP *:omniorb (LISTEN)
docker-pr 3574871            root    4u  IPv6 1001030574      0t0  TCP *:9005 (LISTEN)
docker-pr 3574884            root    4u  IPv6 1001030610      0t0  TCP *:https (LISTEN)
docker-pr 3574898            root    4u  IPv6 1001031245      0t0  TCP *:http (LISTEN)
```

Aunque la información obtenida puede llegar a ser parecida, la perspectiva de lsof es forzosamente desde el interior de la máquina, mientras que nmap puede ser ejecutado desde fuera de la misma, motivo entre otros por el que en ocasiones puede ser utilizada por atacantes en procesos de preexplotación o incluso explotación.

Netstat:

Esta herramienta de línea de comandos muestra un listado de las conexiones activas de una máquina, tanto entrantes como salientes. La información que resulta del uso del comando incluye el protocolo en uso, las tablas de ruteo, las estadísticas de las interfaces y el estado de la conexión.

Algunas de las opciones de netstat:

- **-r, --route** Muestra la tabla de enrutamiento.
- **-i, --interfaces** Muestra la tabla de interfaces
- **-g, --groups** Muestra los miembros del grupo de multidifusión
- **-s, --statistics** Muestra estadísticas de red (como SNMP)
- **-M, --masquerade** Muestra conexiones *enmascaradas*
- **-v, --verbose** Muestra más información en la salida
- **-n, --numeric** No resuelve nombres en general
- **--numeric-hosts** No resuelve el nombre de los hosts
- **--numeric-ports** No resuelve el nombre de los puertos
- **--numeric-users** No resuelve los nombres de usuarios
- **-N, --symbolic** Muestra los nombres del hardware de red
- **-e, --extend** Muestra otra/mas información.

Campus Internacional CIBERSEGURIDAD
ENIIT INNOVA IT BUSINESS SCHOOL
UCAM UNIVERSIDAD CATÓLICA DE MURCIA

- **-p, --programs** Muestra PID o nombre del programa por cada socket
- **-c, --continuous** Muestra continuamente las estadísticas de red (hasta que se interrumpa el programa)
- **-l, --listening** Muestra los *server sockets* que están es modo *escucha*
- **-a, --all, --listening** Muestra todos los sockets (por defecto únicamente los que están en modo *conectado*)
- **-o, --timers** Muestra los *timers*
- **-F, --fib** Muestra el *Forwarding Information Base* (por defecto)
- **-C, --cache** Mostrar el cache de enrutamiento en vez del FIB

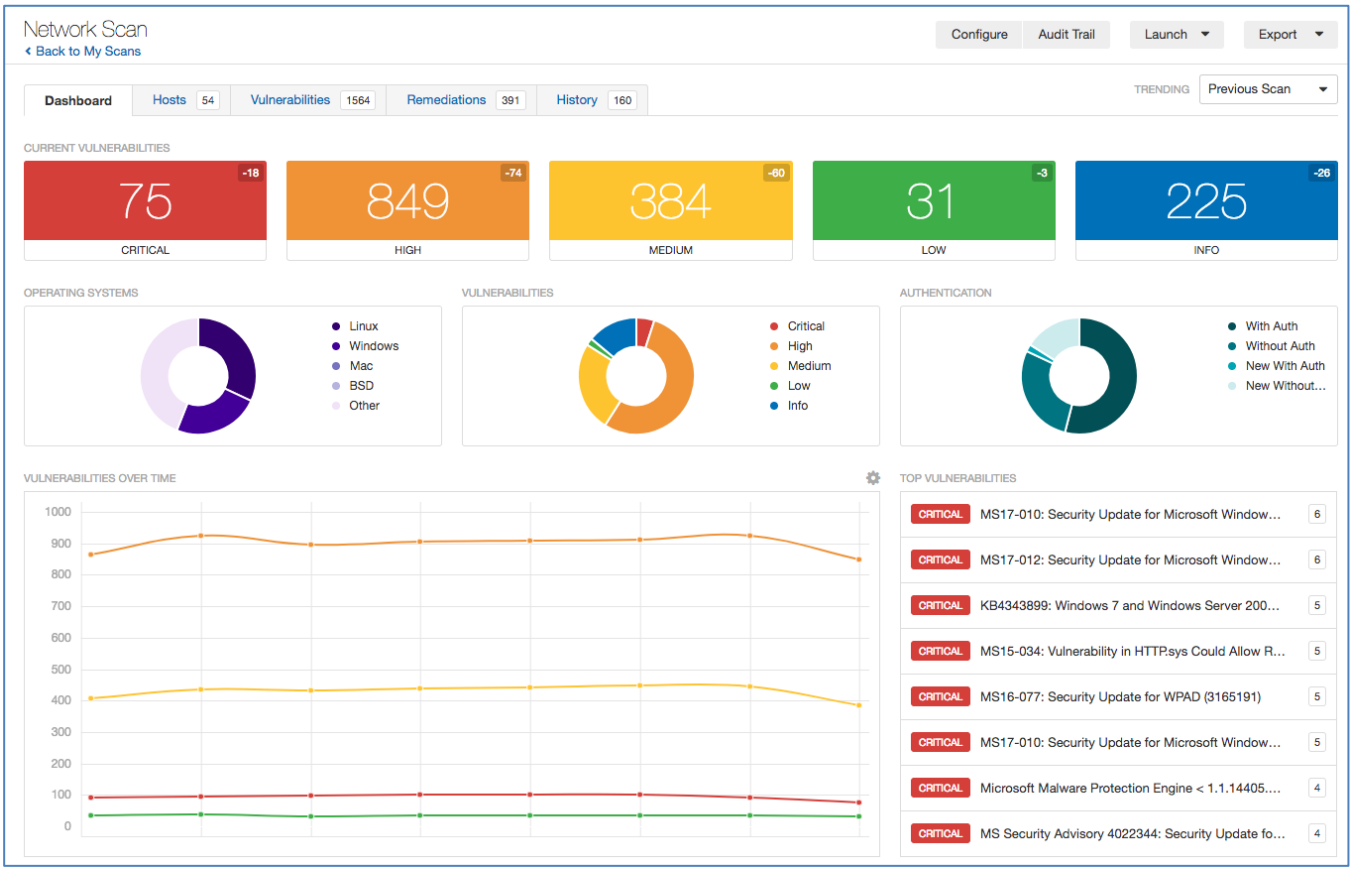
Por ejemplo, el comando **netstat -o 5 -p** nos mostrará por pantalla todas las conexiones TCP abiertas y el id de cada proceso cada 5 segundos:

```
root@rabbitmq-a:/# netstat -o 5 -p
Active Internet connections (w/o servers)
Proto Recv-Q Send-Q Local Address           Foreign Address         State       PID/Program name    Timer
tcp        0      0 localhost:54841         localhost:4369          ESTABLISHED 20/beam.smp         off (0.00/0/0)
tcp        0      0 localhost:50679         localhost:4369          TIME_WAIT   -                   timewait (38.03/0/0)
tcp        0      0 rabbitmq-a:4369         localhost:44423         TIME_WAIT   -                   timewait (37.86/0/0)
tcp        0      0 localhost:4369          localhost:44441         ESTABLISHED 79/epmd             off (0.00/0/0)
tcp6       0      0 rabbitmq-a:5672         [illegible]99:37500     ESTABLISHED 20/beam.smp         off (0.00/0/0)
tcp6       0      0 rabbitmq-a:5672         [illegible]32820        ESTABLISHED 20/beam.smp         off (0.00/0/0)
tcp6       0      0 rabbitmq-a:5672         [illegible]99:37482     ESTABLISHED 20/beam.smp         off (0.00/0/0)
tcp6       0      0 rabbitmq-a:5672         [illegible]e-:54966     ESTABLISHED 20/beam.smp         off (0.00/0/0)
tcp6       0      0 rabbitmq-a:5672         [illegible]99:37474     ESTABLISHED 20/beam.smp         off (0.00/0/0)
tcp6       0      0 rabbitmq-a:5672         [illegible]32830        ESTABLISHED 20/beam.smp         off (0.00/0/0)
tcp6       0      0 rabbitmq-a:5672         [illegible]32858        ESTABLISHED 20/beam.smp         off (0.00/0/0)
tcp6       0      0 rabbitmq-a:5672         [illegible]99:37484     ESTABLISHED 20/beam.smp         off (0.00/0/0)
tcp6       0      0 rabbitmq-a:5672         [illegible]e-:54956     ESTABLISHED 20/beam.smp         off (0.00/0/0)
tcp6       0      0 rabbitmq-a:5672         [illegible]99:37468     ESTABLISHED 20/beam.smp         off (0.00/0/0)
tcp6       0      0 rabbitmq-a:5672         [illegible]e-:54964     ESTABLISHED 20/beam.smp         off (0.00/0/0)
tcp6       0      0 rabbitmq-a:5672         [illegible]99:37476     ESTABLISHED 20/beam.smp         off (0.00/0/0)
tcp6       0      0 rabbitmq-a:5672         [illegible]t-s:46416    ESTABLISHED 20/beam.smp         off (0.00/0/0)
tcp6       0      0 rabbitmq-a:5672         [illegible]e-:54952     ESTABLISHED 20/beam.smp         off (0.00/0/0)
tcp6       0      0 rabbitmq-a:5672         [illegible]32832        ESTABLISHED 20/beam.smp         off (0.00/0/0)
tcp6       0      0 rabbitmq-a:5672         [illegible]99:37472     ESTABLISHED 20/beam.smp         off (0.00/0/0)
tcp6       0      0 rabbitmq-a:5672         [illegible]32824        ESTABLISHED 20/beam.smp         off (0.00/0/0)
tcp6       0      0 rabbitmq-a:5672         [illegible]32822        ESTABLISHED 20/beam.smp         off (0.00/0/0)
tcp6       0      0 rabbitmq-a:5672         [illegible]e-:54968     ESTABLISHED 20/beam.smp         off (0.00/0/0)
tcp6       0      0 rabbitmq-a:5672         [illegible]e-:54962     ESTABLISHED 20/beam.smp         off (0.00/0/0)
tcp6       0      0 rabbitmq-a:5672         [illegible]t-s:46422    ESTABLISHED 20/beam.smp         off (0.00/0/0)
tcp6       0      0 rabbitmq-a:5672         [illegible]e-:54958     ESTABLISHED 20/beam.smp         off (0.00/0/0)
tcp6       0      0 rabbitmq-a:5672         [illegible]32836        ESTABLISHED 20/beam.smp         off (0.00/0/0)
tcp6       0      0 rabbitmq-a:5672         [illegible]t-s:46414    ESTABLISHED 20/beam.smp         off (0.00/0/0)
tcp6       0      0 rabbitmq-a:5672         [illegible]32826        ESTABLISHED 20/beam.smp         off (0.00/0/0)
tcp6       0      0 rabbitmq-a:5672         [illegible]99:37470     ESTABLISHED 20/beam.smp         off (0.00/0/0)
tcp6       0      0 rabbitmq-a:5672         [illegible]99:37480     ESTABLISHED 20/beam.smp         off (0.00/0/0)
tcp6       0      0 rabbitmq-a:5672         [illegible]e-:54954     ESTABLISHED 20/beam.smp         off (0.00/0/0)
tcp6       0      0 rabbitmq-a:5672         [illegible]e-:54960     ESTABLISHED 20/beam.smp         off (0.00/0/0)
tcp6       0      0 rabbitmq-a:5672         [illegible]32834        ESTABLISHED 20/beam.smp         off (0.00/0/0)
tcp6       0      0 rabbitmq-a:5672         [illegible]t-s:46424    ESTABLISHED 20/beam.smp         off (0.00/0/0)
tcp6       0      0 rabbitmq-a:5672         [illegible]99:37478     ESTABLISHED 20/beam.smp         off (0.00/0/0)
Active UNIX domain sockets (w/o servers)
Proto RefCnt Flags       Type       State         I-Node   PID/Program name     Path
unix  3      [ ]         STREAM     CONNECTED     1001030181 49/erl_child_setup
unix  3      [ ]         STREAM     CONNECTED     1001030180 20/beam.smp
```

Nessus:

Además de herramientas por línea de comandos, también existen soluciones más potentes, aunque también mas complejas (y en su mayoría comerciales) que permiten realizar escaneos (usando entre otras herramientas nmap) y análisis de vulnerabilidades masivos incluso en redes completas. Nessus es un ejemplo de ellas, que permite lanzar multiples escaneos masivos y observar las estadísticas en un panel de administración centralizado.

Esta herramienta permite, por ejemplo, la planificación de escaneos periódicos e incluso intentos de ejecución de exploits (similar a los scripts de nmap) para intentar explotar los puertos abiertos detectados.



Ejemplo de visualización del resultado de un escaneo masivo desde el panel de administración de Nessus

4. USO DE LAS LLAMADAS DEL KERNEL QUE HACEN LAS APPS

4.1. STRACE

La herramienta de línea de comandos **strace**, diseñada para GNU/Linux permite comprobar errores en el sistema operativo y en las aplicaciones que se ejecutan en él. A través de strace, es posible vigilar las llamadas al sistema que ejecuta un determinado programa, así como mostrar todas las señales que este recibe desde el SO durante su ejecución. Esto es posible a su vez gracias a la llamada al sistema del núcleo de Linux llamada **ptrace**.

4.1.1. Uso de strace

El uso más común de strace es, arrancarlo junto al programa que se desea trazar, lo que imprimirá una lista de las llamadas al sistema operativo que este ejecuta, así como los posibles errores que puedan surgir (incluyendo la salida del propio programa). Gracias a esto, es posible detectar situaciones de error como, por ejemplo, que el programa intente acceder a un fichero o recurso que no existe o para el que no tiene permiso de acceso.

Ejecución y salida de strace sobre el comando “date”, que imprime la fecha y hora actual del sistema:

Campus Internacional CIBERSEGURIDAD
ENIIT INNOVA IT BUSINESS SCHOOL
UCAM UNIVERSIDAD CATÓLICA DE MURCIA

```
admin1@mru8: ~
admin1@mru8:~$ strace date
execve("/bin/date", ["date"], 0x7ffdc9f0df20 /* 22 vars */) = 0
brk(NULL)                               = 0x555a10853000
access("/etc/ld.so.nohwcap", F_OK)      = -1 ENOENT (No such file or director
y)
access("/etc/ld.so.preload", R_OK)      = -1 ENOENT (No such file or director
y)
openat(AT_FDCWD, "/etc/ld.so.cache", O_RDONLY|O_CLOEXEC) = 3
fstat(3, {st_mode=S_IFREG|0644, st_size=22758, ...}) = 0
mmap(NULL, 22758, PROT_READ, MAP_PRIVATE, 3, 0) = 0x7fe6a39f3000
close(3)                                = 0
access("/etc/ld.so.nohwcap", F_OK)      = -1 ENOENT (No such file or director
y)
openat(AT_FDCWD, "/lib/x86_64-linux-gnu/libc.so.6", O_RDONLY|O_CLOEXEC) = 3
read(3, "\177ELF\2\1\1\3\0\0\0\0\0\0\0\0\3\0>\0\1\0\0\0\260\34\2\0\0\0\0\0"..
., 832) = 832
fstat(3, {st_mode=S_IFREG|0755, st_size=2030544, ...}) = 0
mmap(NULL, 8192, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) = 0x
7fe6a39f1000
mmap(NULL, 4131552, PROT_READ|PROT_EXEC, MAP_PRIVATE|MAP_DENYWRITE, 3, 0) = 0
x7fe6a33e1000
mprotect(0x7fe6a35c8000, 2097152, PROT_NONE) = 0
mmap(0x7fe6a37c8000, 24576, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_FIXED|MAP_D
ENYWRITE, 3, 0x1e7000) = 0x7fe6a37c8000
mmap(0x7fe6a37ce000, 15072, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_FIXED|MAP_A
NONYMOUS, -1, 0) = 0x7fe6a37ce000
close(3)                                = 0
arch_prctl(ARCH_SET_FS, 0x7fe6a39f2540) = 0
mprotect(0x7fe6a37c8000, 16384, PROT_READ) = 0
mprotect(0x555a0f339000, 8192, PROT_READ) = 0
mprotect(0x7fe6a39f9000, 4096, PROT_READ) = 0
munmap(0x7fe6a39f3000, 22758)           = 0
brk(NULL)                               = 0x555a10853000
brk(0x555a10874000)                     = 0x555a10874000
openat(AT_FDCWD, "/usr/lib/locale/locale-archive", O_RDONLY|O_CLOEXEC) = 3
fstat(3, {st_mode=S_IFREG|0644, st_size=1683056, ...}) = 0
mmap(NULL, 1683056, PROT_READ, MAP_PRIVATE, 3, 0) = 0x7fe6a3856000
close(3)                                = 0
openat(AT_FDCWD, "/etc/localtime", O_RDONLY|O_CLOEXEC) = 3
fstat(3, {st_mode=S_IFREG|0644, st_size=127, ...}) = 0
fstat(3, {st_mode=S_IFREG|0644, st_size=127, ...}) = 0
read(3, "TZif2\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\1\0\0\0\1\0\0\0\0"..., 409
6) = 127
lseek(3, -71, SEEK_CUR)                 = 56
read(3, "TZif2\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\1\0\0\0\1\0\0\0\0"..., 409
6) = 71
close(3)                                = 0
fstat(1, {st_mode=S_IFCHR|0620, st_rdev=makedev(136, 0), ...}) = 0
write(1, "Sun Feb 16 12:25:49 UTC 2020\n", 29Sun Feb 16 12:25:49 UTC 2020
) = 29
close(1)                                = 0
close(2)                                = 0
exit_group(0)                           = ?
+++ exited with 0 +++
```

4.1.2. Filtrando llamadas concretas al sistema

En ocasiones strace puede ser muy "verbose" y dar más información de la que necesitamos, dificultando la tarea de identificar las llamadas que nos interesan. Así, si solo nos interesa un tipo especifico de llamadas al sistema, podemos utilizar la opción **-etrace={funcion}** en el lanzamiento de strace. Por ejemplo, para la llamada anterior, si solo nos interesa inspeccionar los ficheros que son abiertos por el programa, lo lanzaremos de la siguiente forma:

strace -etrace=openat date

Nota: se usa open(at) porque, a pesar de tener un funcionamiento muy similar a "open", aquellas llamadas al sistema con el sufijo "at" aportan un par de diferencias sobre las "básicas".

- Evitan condiciones de carrera en el acceso al recurso.
- Permiten la implementación de un CWD por hebra mediante un descriptor de fichero mantenido por la propia aplicación. Esto evita colisiones y problemas de concurrencia a la hora de obtener el descriptor de fichero desde distintas hebras.

Esto generará la siguiente salida:

```
admin1@mru8:~$ strace -etrace=openat date
openat(AT_FDCWD, "/etc/ld.so.cache", O_RDONLY|O_CLOEXEC) = 3
openat(AT_FDCWD, "/lib/x86_64-linux-gnu/libc.so.6", O_RDONLY|O_CLOEXEC) = 3
openat(AT_FDCWD, "/usr/lib/locale/locale-archive", O_RDONLY|O_CLOEXEC) = 3
openat(AT_FDCWD, "/etc/localtime", O_RDONLY|O_CLOEXEC) = 3
Sun Feb 16 13:06:15 UTC 2020
+++ exited with 0 +++
```

4.1.3. Algunas syscall interesantes a tener en cuenta a la hora de usar strace

A continuación, se muestran algunas llamadas al sistemas que nos puede ser útil filtrar mientras depuramos una aplicación:

- **execve**: ejecución de programas. La primera línea de strace suele comenzar con la llamada a execve para lanzar el programa que estamos depurando.
- **open(at):** Apertura de ficheros.
- **connect:** Apertura de conexiones remotas (normalmente internet).
  Ejemplo: Uso de curl.

Campus Internacional CIBERSEGURIDAD
ENIIT INNOVA IT BUSINESS SCHOOL
UCAM UNIVERSIDAD CATÓLICA DE MURCIA

```
admin1@mru8:~$ strace -etrace=connect curl google.com
connect(3, {sa_family=AF_INET, sin_port=htons(80), sin_addr=inet_addr("172.21
7.17.14")}, 16) = -1 EINPROGRESS (Operation now in progress)
<HTML><HEAD><meta http-equiv="content-type" content="text/html;charset=utf-8"
>
<TITLE>301 Moved</TITLE></HEAD><BODY>
<H1>301 Moved</H1>
The document has moved
<A HREF="http://www.google.com/">here</A>.
</BODY></HTML>
+++ exited with 0 +++
```

- **sendto:** Envía un mensaje vía socket (https://linux.die.net/man/2/sendto)

```
admin1@mru8:~$ strace -etrace=sendto curl google.com
sendto(3, "GET / HTTP/1.1\r\nHost: google.com"..., 74, MSG_NOSIGNAL, NULL, 0)
 = 74
<HTML><HEAD><meta http-equiv="content-type" content="text/html;charset=utf-8"
>
<TITLE>301 Moved</TITLE></HEAD><BODY>
<H1>301 Moved</H1>
The document has moved
<A HREF="http://www.google.com/">here</A>.
</BODY></HTML>
+++ exited with 0 +++
```

- **recvfrom:** Recive un mensaje desde socket (https://linux.die.net/man/2/recvfrom)
- **read/write:** Lectura/escritura sobre un descriptor de fichero

5. REESCRITURA DE MÓDULOS DEL KERNEL PARA EXTRAER INFORMACIÓN DEL SISTEMA

Ya que depurar a bajo nivel el sistema operativo, ya sea tanto en modo kernel como usuario (aplicaciones) puede ser complicado (incluso si hablamos únicamente de un módulo), una de las opciones más asequibles para obtener información de lo que está pasando es reemplazar los módulos originales del kernel para incluir nuevos modificados por nosotros mismos con el objetivo de imprimir o exportar información extra que pueda ser de utilidad en la depuración del sistema. A esta técnica se le conoce como "Debugging by Printing" o "Depuración por Impresión" y básicamente consiste en la introducción de "prints" (**printk** en el caso del kernel) en el código a depurar con el objetivo de ver en claro qué esta ocurriendo.

Mas info: https://elinux.org/Debugging_by_printing

5.1. GESTIÓN DE MÓDULOS DEL KERNEL

Como ya se ha explicado en la unidad anterior, el kernel de Linux tiene un diseño modular. En el momento del arranque, solo se carga un kernel residente mínimo en memoria. De este modo, conforme un usuario solicita alguna característica que no esta presente en le kernel residente, si es posible, esta es cargada dinámicamente en memoria mediante un módulo kernel. Recordemos que, tanto el kernel como los módulos asociados correrán siempre en memoria. De hecho, en concreto el kernel es cargado en memoria en el momento de arranque del sistema.

Linux incluye un grupo de comandos para el manejo de modulos kernel (siempre que el paquete **modutils** esté instalado). Gracias a estos comandos, es posible realizar distintas tareas como, por ejemplo, la carga de un nuevo módulo o comprobar si un módulo se cargado exitosamente.

5.1.1. Listado de los módulos cargados en el kernel

Podemos consultar los módulos extra instalados en nuestro sistema mediante una consulta al fichero **/etc/modules**. Como se comentó en la unidad 1, este fichero contiene el listado de nombres de los módulos que deben ser cargados en boot time, es decir, durante el arranque del sistema. Adicionalmente, podemos consultar qué modulos están cargados actualmente en el kernel usando el comando /sbin/lsmod, que muestra una lista del os módulos cargados. Esto es equivalente a la salida que obtendremos si ejecutamos un listado del fichero **/proc/modules.**

Un ejemplo de listado de módulos sobre Ubuntu 18.04 arrojaría una salida similar a la siguiente:

Campus Internacional CIBERSEGURIDAD
ENIIT INNOVA IT BUSINESS SCHOOL
UCAM UNIVERSIDAD CATÓLICA DE MURCIA

```
admin1@mru8:/proc$ lsmod
Module                  Size  Used by
vmw_balloon            20480  0
intel_rapl_perf        16384  0
vmw_vsock_vmci_transport    32768  1
vsock                  36864  2 vmw_vsock_vmci_transport
joydev                 24576  0
input_leds             16384  0
serio_raw              16384  0
snd_ens1371            28672  0
snd_ac97_codec        131072  1 snd_ens1371
gameport               20480  1 snd_ens1371
btusb                  45056  0
btrtl                  16384  1 btusb
btbcm                  16384  1 btusb
uvcvideo               86016  0
videobuf2_vmalloc      16384  1 uvcvideo
btintel                16384  1 btusb
snd_rawmidi            32768  1 snd_ens1371
videobuf2_memops       16384  1 videobuf2_vmalloc
videobuf2_v4l2         24576  1 uvcvideo
snd_seq_device         16384  1 snd_rawmidi
ac97_bus               16384  1 snd_ac97_codec
videobuf2_core         40960  2 videobuf2_v4l2,uvcvideo
bluetooth             548864  5 btrtl,btintel,btbcm,btusb
snd_pcm                98304  2 snd_ac97_codec,snd_ens1371
snd_timer              32768  1 snd_pcm
videodev              184320  3 videobuf2_core,videobuf2_v4l2,uvcvideo
ecdh_generic           24576  1 bluetooth
snd                    81920  6 snd_seq_device,snd_timer,snd_ac97_codec
media                  40960  2 videodev,uvcvideo
soundcore              16384  1 snd
vmw_vmci               69632  2 vmw_balloon,vmw_vsock_vmci_transport
shpchp                 36864  0
```

Cada línea de la salida corresponde a un módulo, para el que se muestran tres columnas, que corresponden respectivamente con: nombre, tamaño y el recuento de usos registrados para el proceso.

Para obtener información detallada de un modulo presente en el sistema, podemos ejecutar:

/sbin/modinfo [opciones]<nombre del módulo>

En el caso del módulo shpchp por ejemplo, obtendremos una salida similar a la siguiente:

```
admin1@mru8:/proc$ modinfo shpchp
filename:       /lib/modules/4.15.0-76-generic/kernel/drivers/pci/hotplug/shpchp.ko
license:        GPL
description:    Standard Hot Plug PCI Controller Driver
author:         Dan Zink <dan.zink@compaq.com>, Greg Kroah-Hartman <greg@kroah.com>, Dely Sy <dely.l.sy@
intel.com>
srcversion:     9553B71E7C38471DF0A0BA1
alias:          pci:v*d*sv*sd*bc06sc04i00*
depends:
retpoline:      Y
intree:         Y
name:           shpchp
vermagic:       4.15.0-76-generic SMP mod_unload
signat:         PKCS#7
signer:
sig_key:
sig_hashalgo:   md4
parm:           shpchp_debug:Debugging mode enabled or not (bool)
parm:           shpchp_poll_mode:Using polling mechanism for hot-plug events or not (bool)
parm:           shpchp_poll_time:Polling mechanism frequency, in seconds (int)
```

Adicionalmente podremos usar las opciones **-d**, que muestra una breve descripción del módulo y **-p**, que lista los parámetros que soporta el módulo. Consultar la página del manual (man modinfo) para más información sobre el resto de opciones disponibles.

En el caso concreto del módulo shpchp, es interesante subrayar la existencia del parámetro **shpchp_debug**, que habilita la depuración del mismo. Si necesitamos depurar algún módulo, es importante tener presente esta posibilidad presente para algunos módulos.

5.1.2. Descarga de módulos del kernel

Si un módulo no ha sido usado, mostrará el valor 0 para la última columna (Used by). Esto hace posible que podamos descargar el módulo si lo estimamos necesario mediante el comando **rmmod <nombre del módulo>.** Así por ejemplo, ante la salida anterior de **lsmod**, podríamos utilizar rmmod shpchp para descargar el módulo **shpchp** que no ha sido usado por el sistema sin ningún problema. Esta operación requiere permisos de root.

Alternativamente a rmmod podemos usar el comando **modprobe -r <nombre del módulo>**, que descargará el módulo y adicionalmente intentará descargar aquellas dependencias que dejen de ser usadas.

Campus Internacional CIBERSEGURIDAD
ENIIT INNOVA IT BUSINESS SCHOOL
UCAM UNIVERSIDAD CATÓLICA DE MURCIA

5.1.3. Carga de módulos del kernel

Para cargar un módulo del kernel, es posible usar dos comandos:

- **/sbin/modprobe <nombre del módulo>**
- **/sbin/insmod <nombre del módulo>**

Aunque hay que tener en cuenta que **insmod** no resuelve dependencias, por lo que se recomienda usar el primero. A bajo nivel, modprobe usa insmod.

Por defecto, la carga de módulos del kernel intenta cargar el módulo desde los subdirectorios **/lib/modules/<kernel-version>/kernel/drivers/,** donde para cada tipo de módulo existirá un subdirectorio. La siguiente captura muestra un listado de los distintos tipos incluidos en la versión del kernel 4.15.0.

```
admin1@mru8:/proc$ ls /lib/modules/4.15.0-76-generic/kernel/drivers/
acpi/        bluetooth/   dma/         fsi/         hwtracing/   isdn/        media/       net/
pcmcia/      ptp/         rtc/         thermal/     vhost/
ata/         char/        edac/        gpio/        i2c/         leds/        memstick/    nfc/
phy/         pwm/         scsi/        thunderbolt/ video/
atm/         clk/         extcon/      gpu/         iio/         lightnvm/    message/     ntb/
pinctrl/     rapidio/     spi/         tty/         virtio/
auxdisplay/  cpufreq/     firewire/    hid/         infiniband/  macintosh/   mfd/         nvdimm/
platform/    regulator/   spmi/        uio/         vme/
base/        crypto/      firmware/    hsi/         input/       mailbox/     misc/        nvme/
power/       remoteproc/  ssb/         usb/         w1/
bcma/        dax/         fmc/         hv/          iommu/       mcb/         mmc/         parport/
powercap/    reset/       staging/     uwb/         watchdog/
block/       dca/         fpga/        hwmon/       ipack/       md/          mtd/         pci/
pps/         rpmsg/       target/      vfio/        xen/
```

Así por ejemplo, si quisiéramos volver a cargar el módulo del kernel shpchp que previamente hemos descargado, podríamos ejecutar modprobe shpchp, que podemos encontrar en:

/lib/modules/4.15.0-76-generic/kernel/drivers/pci/hotplug/shpchp.ko

Como se puede observar, los módulos del kernel tienen extensión **.ko** (kernel object), que desde la versión 2.6 del kernel, ha reemplazado a los ficheros con extensión .o .

Campus Internacional CIBERSEGURIDAD
ENIIT INNOVA IT BUSINESS SCHOOL
UCAM UNIVERSIDAD CATÓLICA DE MURCIA

Si durante la instalación ocurre algún problema, podemos usar modprobe con la opción **-v**, que imprimirá por pantalla todos los comandos que se ejecutan durante el proceso de carga y puede se especialmente útil en la detección de problemas con alguno de los módulos que intenta cargar nuestro sistema.

5.1.4. Configuración de opciones de los módulos del kernel

Una opción rápida para conocer la configuración actual de un módulo es ejecutar el comando:

modprobe -c | grep nombre_del_módulo

La salida, será equivalente a la siguiente en el caso del módulo shpchp que estamos usando como ejemplo:

```
admin1@mru8:/proc$ modprobe -c | grep shpchp
alias pci:v*d*sv*sd*bc06sc04i00* shpchp
```

Las opciones del módulo pueden ser modificadas editando (o creando si es necesario) el fichero <nombre del módulo>.conf en dentro del directorio /etc/modprobe.d/, que se usa para pasar la configuración de los módulos a **udev** (el gestor de dispositivos usado por el kernel de Linux). La sintaxis dentro del fichero, será la siguiente:

```
/etc/modprobe.d/nombredelmodulo.conf

options nombredelmodulo nombredelparametro=valordelparametro
```

De este modo, por ejemplo, si quieremos habilitar la depuración en el módulo shpchp, necesitaremos crear el fichero shpchp.conf con el siguiente contenido:

```
/etc/modprobe.d/shpchp.conf

options shpchp shpchp_debug=1
```

Una vez reinstalado el módulo, si volvemos a consultar la información del mismo, obtendremos la siguiente salida que indica que, efectivamente, el módulo se ha cargado con la opción de depuración activa:

```
admin1@mru8:/proc$ modprobe -c | grep shpchp
options shpchp shpchp_debug=1
alias pci:v*d*sv*sd*bc06sc04i00* shpchp
```

Es importante tener en cuenta que esta modificación es persistente, por lo que será tenido en cuenta cada vez que inicie el sistema y se intente cargar el módulo.

5.2. ESCRITURA E INSTALACIÓN DE UN MÓDULO DEL KERNEL

Por lo general, los módulos del sistema están escritos íntegramente en C.

Conocimientos necesarios:

- Programación C
- Make: Invocación…
- Makefile
- GCC: Compilador C/C++

5.2.1. Escritura y compilación de nuestro módulo

Antes de nada, instalamos las cabeceras y dependencias básicas para poder escribir módulos de nuestra versión (uname -r) de kernel de Linux.

```
sudo apt-get install build-essential linux-headers-$(uname -r)
```

Una vez hecho esto, podremos comenzar a desarrollar nuestro módulo. En este caso, se trata de un sencillo módulo Hello World que únicamente imprime un mensaje característico durante su instalación y desinstalación. Lo llamaremos “my_first_module.c”.

```
//Cabeceras básicas para trabajar con módulos
#include <linux/module.h>  // included for all kernel modules
#include <linux/kernel.h>  // included for KERN_INFO
#include <linux/module.h>  // included for __init and __exit macros

MODULE_LICENSE("GPL");
MODULE_AUTHOR("MRU8");
MODULE_DESCRIPTION("A simple Hello World kernel module");

static int __init hello_init(void){
        printk(KERN_INFO "Hello World!\n");
        return 0;
        // Si el método devolviera un resultado distinto a cero, significaría que no puede ser cargado
}
```

5.2.2. Instalación de nuestro módulo en el kernel

Mas información interesante sobre los módulos de Linux: https://wiki.archlinux.org/index.php/Kernel_module_(Espa%C3%B1ol)

Una vez creado el código fuente de nuestro módulo, el siguiente paso será definir las rutinas de gestión de la compilación de mismo utilizando el programa make. Para ello, crearemos el fichero "Makefile" (este nombre es el usado por defecto para ser usado por el comando make). En este

fichero definiremos las tareas a ejecutar tanto en la compilación, resolución de dependencias, recompilación y limpieza, etc. El contenido de nuestro makefile será el siguiente:

```
# Con la opción -C añadimos el directorio de trabajo /lib/modules/$(shell uname -r)/build
# donde están las dependencias que necesitamos para compilar

obj-m += my_first_kmodule.o

all:
	make -C /lib/modules/$(shell uname -r)/build M=$(shell pwd) modules

clean:
```

* Nota: Para make es especialmente importante el uso de tabulaciones. Es necesario tener en cuenta que, en este caso, el uso de grupos de espacios no equivale a un tabulador.

La inclusión de **M=$(shell pwd)** implica la definición de una variable en la ejecución que indica a make que debe construir el módulo en la ruta indicada por **pwd**, es decir, el directorio de trabajo actual al que regresará tras resolver las dependencias indicadas mediante -C.

Una vez disponemos de los ficheros my_fisrt_kmodule.c y Makefile, podemos ejecutar el comando **make** para compilar. Si tenemos problemas podemos ejecutar **make --debug** para ver más detalladamente qué puede estar ocurriendo.

Durante la ejecución, si todo funciona correctamente obtendremos una salida similar a la siguiente:

```
root@mru8:/home/admin1/my_first_kmodule# ls
Makefile  my_first_kmodule.c
root@mru8:/home/admin1/my_first_kmodule# make
make -C /lib/modules/4.15.0-76-generic/build M=/home/admin1/my_first_kmodule
modules
make[1]: Entering directory '/usr/src/linux-headers-4.15.0-76-generic'
  CC [M]  /home/admin1/my_first_kmodule/my_first_kmodule.o
/home/admin1/my_first_kmodule/my_first_kmodule.c:17:20: warning: 'hello_clean
up' defined but not used [-Wunused-function]
 static void __exit hello_cleanup(void){
                    ^~~~~~~~~~~~~
/home/admin1/my_first_kmodule/my_first_kmodule.c:11:19: warning: 'hello_init'
 defined but not used [-Wunused-function]
 static int __init hello_init(void){
                   ^~~~~~~~~~
  Building modules, stage 2.
  MODPOST 1 modules
  CC      /home/admin1/my_first_kmodule/my_first_kmodule.mod.o
  LD [M]  /home/admin1/my_first_kmodule/my_first_kmodule.ko
make[1]: Leaving directory '/usr/src/linux-headers-4.15.0-76-generic'
```

Como se puede observar, make básicamente está entrando al directorio /lib/modules/{versión del kernel}/build para obtener las cabeceras necesarias. Posteriormente, construye nuestro módulo, avisando de que hemos declarado dos funciones no utilizadas y por último genera una serie de ficheros asociados al mismo para despues salir del directorio y finalizar su ejecución.

```
root@mru8:/home/admin1/my_first_kmodule# ls
Makefile        my_first_kmodule.c      my_first_kmodule.mod.o
modules.order   my_first_kmodule.ko     my_first_kmodule.o
Module.symvers  my_first_kmodule.mod.c
```

El nuevo módulo, ha sido construido en el fichero my_first_kmodule.ko, que al igual que el resto de modulos presentes en /lib/modules, podremos cargar/descargar usando los comandos explicados anteriormente.

5.2.3. Instalación/desinstalación de un módulo propio

Al igual que se explicó en el apartado anterior, podemos cargar el módulo recién creado usando insmod o modprobe.

Salida de dmesg | tail -10:

```
peration="profile_replace" info="same as current profile, skipping" profile="
unconfined" name="snap.core.hook.configure" pid=13464 comm="apparmor_parser"
[95581.439198] Hello World!
[95589.684593] Cleaning up module.
root@mru8:/home/admin1/my_first_kmodule#
```

Salida de tail /var/log/syslog:

```
Feb 23 16:22:05 mru8 systemd-timesyncd[591]: Synchronized to time server 91.1
89.89.199:123 (ntp.ubuntu.com).
Feb 23 16:26:16 mru8 kernel: [95581.439198] Hello World!
Feb 23 16:26:25 mru8 kernel: [95589.684593] Cleaning up module.
```

Como se puede observar en la imagen superior el módulo ha impreso los mensajes esperados tanto en la carga como en la descarga.

Mas información sobre manejo de módulos kernel: https://wiki.archlinux.org/index.php/Kernel_module

6. DEPURACIÓN DE LA ACTIVIDAD DEL KERNEL

En esta sección se utilizarán algunas de las técnicas aplicadas en las anteriores para recoger toda la información que permiten extraer y relacionarla para tener una visión mas amplia de la actividad del kernel que nos permita su depuración.

6.1. LA HERRAMIENTA DMESG

Es una herramienta por linea de commandos que permite leer "en crudo" el buffer de mensajes del kernel. Estos mensajes incluyen aquellos generados durante el arranque del sistema, detección de hardware, asignación de controladores (drivers) y la propia depuración de aplicaciones. El demonio syslog persiste la información generada por dmesg en disco en formato log **(/var/log/syslog**)

6.2. CONFIGURACIÓN DE LOS NIVELES DE LOG DEL KERNEL

Como cualquier otro programa, el kernel utiliza mensajes de log para devolver información sobre su funcionamiento. Estos logs, estarán categorizados según distintos niveles de importancia y propósito. Así por ejemplo, habrá mensajes que irán desde nivel debug, a hasta nivel de emergencia (cuando el sistema se ha vuelto insestable e incluso corre riesgo de colapso),

pasando por otros como por ejemplo warning o error. Sin embargo, según la configuración del sistema operativo, al consumir los distintos puntos de acceso a estos logs, este solo mostrará aquellos que por nivel de importancia superen al umbral establecido como mínimo.

La siguiente tabla representa los distintos tipos de mensajes de log que puede imprimir el kernel así como sus niveles de importancia que van del 0 al 7 siendo el 0 el de mayor nivel.

Name	String	Meaning	alias function
KERN_EMERG	"0"	Emergency messages, system is about to crash or is unstable	pr_emerg
KERN_ALERT	"1"	Something bad happened and action must be taken immediately	pr_alert
KERN_CRIT	"2"	A critical condition occurred like a serious hardware/software failure	pr_crit
KERN_ERR	"3"	An error condition, often used by drivers to indicate difficulties with the hardware	pr_err
KERN_WARNING	"4"	A warning, meaning nothing serious by itself but might indicate problems	pr_warning
KERN_NOTICE	"5"	Nothing serious, but notably nevertheless. Often used to report security events.	pr_notice
KERN_INFO	"6"	Informational message e.g. startup information at driver initialization	pr_info
KERN_DEBUG	"7"	Debug messages	pr_debug, pr_devel if DEBUG is defined

Campus Internacional CIBERSEGURIDAD
ENIIT INNOVA IT BUSINESS SCHOOL
UCAM UNIVERSIDAD CATÓLICA DE MURCIA

KERN_DEFAULT	"d"	The default kernel loglevel	
KERN_CONT	""	"continued" line of log printout (only done after a line that had no enclosing \n) [1]	pr_cont

Además de los niveles del 0 al 7, existen dos modos adicionales KERN_DEFAULT (el mas habitualmente utilizado en el modo normal del kernel) y KERN_CONT, utilizado para continuar una línea anterior que no ha sido terminada con un salto de línea.

Las funciones mostradas en la columna "alias_function" reciben como parámetro el mensaje a imprimir y automáticamente incluirán el nivel de log correspondiente.

Según el nivel de log de cada mensaje, el kernel podrá mostrar de forma inmediata el mensaje al usuario por consola (lo que incluye puerto serie, impresora, etc) o no. Esta selección de niveles mínimos de impresión de mensajes se representa a través de la variable del kernel ***console_loglevel***, de esta forma, si un mensaje supera el nivel mínimo definido por esta variable, será mostrado en la consola actual.

Para determinar qué valor de ***console_loglevel*** se está usando bastará con ejecutar:

```
$ cat /proc/sys/kernel/printk
  4     4     1     7
  current default minimum        boot-time-default
```

Como se puede observar en la salida anterior, en el momento de ejecutar el comando, el nivel mínimo de log es 4 (WARNING), que también es el por defecto y será aplicado a aquellos mensajes que no indiquen nivel de importancia, siendo 1 (el valor mínimo que se puede definir en esta variable, es decir, el mayor justo por debajo de KERN_EMERG que solo será utilizado en casos extremos. Por último, se indica que el nivel 7 será default_console_loglevel, es decir, el nivel por defecto en el momento de arranque del sistema.

Más información: https://linuxconfig.org/introduction-to-the-linux-kernel-log-levels

De cara a obtener la mayor información del kernel sin alterarlo, una opción es bajar el valor mínimo de ***console_loglevel*** hasta DEBUG, con lo que será posible obtener mensajes mucho más informativos que los habituales y destinados por definición a la depuración de la actividad del kernel. Una forma de modificar este valor, es escribir directamente en /proc/sys/kernel/printk:

```
# echo 8 > /proc/sys/kernel/printk
```

Ahora, por defecto se mostrarán todos los mensajes que imprima el kernel. Si usamos la herramienta **dmesg**, una opción para conseguir esto es lanzarla indicando con el parámetro -n el nivel mínimo de log a mostrar.

```
# dmesg -n 8
```

El resultado de ambos comandos es equivalente.

6.3. KERNEL TAINTING. "MANCHANDO" EL KERNEL A PROPÓSITO

En la sección anterior se explicó que algunas operaciones sobre el kernel podrían ocasionar su "manchado", como por ejemplo la instalación de módulos desde fuera del árbol de directorios habitual o la instalación de módulos con firmas no reconocidas.

En ocasiones, existen técnicas de obtención de información que, si bien manchan el kernel, son especialmente útiles para su depuración. Una de ellas es la introducción de módulos alterados del sistemas para introducir mensajes de log o printk que nos indique que esta ocurriendo a bajo nivel en el sistema.

Un posible método para depurar controladores o drivers de terceros, sería utilizar strace para conocer qué llamadas del sistema utiliza o qué módulos y modificar estos para mostrar cierta información que nos interese.

Ejemplo interesante de ello: https://www.youtube.com/watch?v=0IQlpFWTFbM&t=3s

Ejercicio:

Utilizando strace + dmesg o syslog (y opcionalmente kernel tainting) depurar la ejecución y encontrar elementos interesanes dentro de una aplicación cualquiera del sistema.

6.4. SYSCALL HOOKING. COMO INTERVENIR LLAMADAS AL SISTEMA

Como ya se ha explicado, el sistema operativo Linux ofrece una serie de llamadas al sistema o syscall que permiten al resto de aplicaciones utilizar funciones del kernel. Las llamadas disponibles, varían en función de la arquitectura del procesador, existiendo así algunas que son específicas de cada arquitectura (32 o 64 bits) y otras que son comunes a ambas, aunque pueda variar su implementación interna.

La API de llamadas al kernel puede verse directamente en el código fuente de Linux. Así, para la versión actual, podremos observarlas directamente en el repositorio GitHub de Linus Torvalds (@torvalds):
https://github.com/torvalds/linux/blob/master/arch/x86/entry/syscalls/syscall_64.tbl

Dependiendo de la versión del kernel que estemos usando, esta ruta puede variar.

Imaginemos que queremos depurar una aplicación propietaria a cuyo código no tenemos acceso. Como ya hemos visto, además de monitorizar la huella que esta aplicación deja en el sistema (ficheros, logs, etc), es posible conocer un poco más de su funcionamiento inspeccionando qué llamadas al kernel está ejecutando mediante la herramienta strace. Sin embargo, el nivel de detalle no es demasiado

Las técnicas de Hooking son utilizadas tanto para depuración de programas y sistemas operativos (como es nuestro caso) como para la aplicación de funcionalidades e incluso, la ejecución de malware normalmente mediante el empleo de rootkits. En el siguiente bloque se analizarán algunos de los rootkits mas populares. En cuanto al uso de hooking para ampliación de funcionalidades, cabe destacar NetFilter (https://es.wikipedia.org/wiki/Netfilter), que emplea técnicas similares a las que se contarán a continuación para procesar eventos de red dentro del

Campus Internacional CIBERSEGURIDAD
ENIIT INNOVA IT BUSINESS SCHOOL
UCAM UNIVERSIDAD CATÓLICA DE MURCIA

kernel, ya que este framework ha sido integrado como parte del núcleo de linux ofreciendo principalmente funcionalidades de firewall y otras utilidades relacionadas.

Algunos de los procedimientos y técnicas explicadas a continuación serán útiles y deberán ser tenidas en cuenta en la siguiente unidad a la hora de detectar la presencia de rootkits y alteraciones indebidas del sistema.

Si queremos modificar algunos de los módulos del sistema, podemos buscar el código fuente directamente en GitHub:

https://github.com/torvalds/linux/tree/master/drivers

En el caso de la versión del kernel de las máquinas virtuales usadas en esta unidad (v4.15) una posibilidad es bajar directamente esa reléase y navegar hasta el código que nos interesa.

https://github.com/torvalds/linux/releases/tag/v4.15

Es importante buscar el correspondiente a nuestra versión del kernel, ya que si intentamos instalar el incluido en una versión diferente y el código ha cambiado probablemente obtengamos resultados inesperados que incluso puede conllevar un kernel panic.

7. CUSTOMIZANDO EL KERNEL DE LINUX PARA DEPURACIÓN

En la sección anterior, se ha explicado cómo crear e instalar/desinstalar módulos propios en el kernel de Linux. En ocasiones, será posible incluso modificar y sobrescribir módulos preexistentes como por ejemplo ocurre con la mayoría de drivers presentes en el kernel. Sin embargo, esto no siempre es posible, ya que según la versión con la que estemos trabajando, existirán algunos módulos que se habrán incluido directamente en su compilación formando parte como tal del kernel. A esta configuración de los módulos se le conoce como Built-in, es decir, construidos junto con el núcleo.

La edición y carga de módulos Built-in requiere una recompilación completa del kernel que los incluya en el proceso de construcción. Por tanto, será necesario recompilar el núcleo si

Campus Internacional CIBERSEGURIDAD
ENIIT INNOVA IT BUSINESS SCHOOL
UCAM UNIVERSIDAD CATÓLICA DE MURCIA

queremos editar estos módulos para que incluyan el mostrado de información útil para la depuración como por ejemplo printk para mostrar llamadas a funciones, parámetros, errores, etc.

Información oficial y detallada de Ubuntu sobre la recompilación del kernel: https://wiki.ubuntu.com/Kernel/BuildYourOwnKernel

7.1. ESTRUCTURA DEL CÓDIGO FUENTE DEL KERNEL DE LINUX

El código fuente del Kernel de Linux se estructura en varios directorios:

1. **arch**. Este directorio contiene archivos referentes a arquitecturas específicas. Dentro del directorio *arch/* existen subdirectorios para diferentes tipos de arquitecturas como x86, ARM, Motorola 64K, MIPS o SPARC.
2. **block**. Contiene la implementación de algoritmos de planificación de E/S necesarios para el uso de dispositivos de almacenamiento por bloques.
3. **certs**.
4. **crypto**. Contiene la implementación de operaciones de cifrado y la API criptológica que es usada por ejemplo por los dispositivos WiFi para implementar algoritmos de cifrado
5. **Documentation**. Este directorio contiene la descripción de muchos subsistemas, así como información valiosa sobre el funcionamiento del Kernel
6. **drivers**. Este directorio contiene multitud de subdirectorios con los controladores de numerosos dispositivos separados por clases de dispositivo. Es el directorio con mayor contenido de largo
7. **fs**. Contiene la implementación de los diferentes sistemas de archivos como EXT3, EXT4, resiserfs, HFS, XFS, NTFS, FAT y otros
8. **include**. Los ficheros de cabecera del Kernel residen en este subdirectorio.
9. **init**. Contiene código de alto nivel de inicio y arranque
10. **ipc**. Contiene el código de soporte para la Comunicación entre Procesos (IPC) donde se implementan mecanismos como las colas de mensajes, los semáforos, y la memoria compartida
11. **kernel**. Las porciones del kernel independientes de la arquitectura se encuentran en este directorio

12. **lib**. Contiene el código que implementa rutinas de librería
13. **mm**. En este directorio se encuentra la implementación de los mecanismos de gestión de memoria
14. **net**. Contiene la implementación de los protocolos de red.
15. **samples**.
16. **scripts**. Scripts usados durante la construcción del Kernel
17. **security**.
18. **sound**. El subsistema de audio de Linux se encuentra en este subdirectorio
19. **tools**.
20. **usr**. Contiene la implementación del sistema **initramfs**
21. **virt.**

7.2. OBTENCIÓN Y CUSTOMIZADO EL CÓDIGO FUENTE DEL KERNEL

Con el objetivo de customizar el kernel, el primer paso será obtener el código fuente del mismo. Existen multitud de versiones del kernel de Linux alternativas, así como repositorios donde descargarlos. Sin embargo, para evitar incompatibilidades con nuestro sistema operativo y con el objetivo de evitar diferencias significativas con el entorno que deseamos depurar, lo recomendable es descargar la misma versión que esté ejecutando el sistema. Esta versión puede obtenerse con el comando **uname -r**

Una vez obtenida la versión actual, podemos bajar el código directamente desde proyecto oficial de GitHub, en la sección de "releases": https://github.com/torvalds/linux/releases

Para las siguientes demostraciones, usaremos por ejemplo la versión 4.15 (https://github.com/torvalds/linux/releases/tag/v4.15)

Una alternativa a lo anterior para obtener el código de una versión concreta del kernel es hacerlo directamente a través de **apt-get**. Mediante el siguiente comando, se descargaría exactamente la misma versión del kernel que se esté usando actualmente:

```
# apt-get source linux-image-$(uname -r)
```

```
admin1@mru8:~/mycustomcore$ apt-get source linux-image-$(uname -r)
Reading package lists... Done
Picking 'linux-signed' as source package instead of 'linux-image-4.15.0-76-generic'
Need to get 19.2 kB of source archives.
Get:1 http://es.archive.ubuntu.com/ubuntu bionic-updates/main linux-signed 4.15.0-88.88 (dsc) [1,9
49 B]
Get:2 http://es.archive.ubuntu.com/ubuntu bionic-updates/main linux-signed 4.15.0-88.88 (tar) [17.
2 kB]
Fetched 19.2 kB in 0s (68.2 kB/s)
dpkg-source: info: extracting linux-signed in linux-signed-4.15.0
dpkg-source: info: unpacking linux-signed_4.15.0-88.88.tar.xz
admin1@mru8:~/mycustomcore$ ls
linux-signed-4.15.0  linux-signed_4.15.0-88.88.dsc  linux-signed_4.15.0-88.88.tar.xz
admin1@mru8:~/mycustomcore$
```

Una vez descargada, ya sería posible editar aquellos módulos que se deseen utilizar para reportar información. Al tratarse de una compilación que implica el código completo, sería posible modificar además cualquier parte del kernel que se desee, ya que finalmente todo será compilado junto en el mismo proceso.

En el presente ejemplo se modificará el módulo **usbcore**, que, por defecto, en la versión 4.15 ha sido incluido como Built-in y que concentra gran parte de la gestión de dispositivos USB que realiza el sistema. El modulo usbcore se encuentra ubicado en la la ruta del proyecto **/drivers/usb/core/**, siendo el fichero **usb.c** el principal del modulo.

De este modo, al estar configurado como Built-in, no será posible descarcargarlo del kernel, obteniéndose un mensaje de error al intentarlo:

```
$ sudo modprobe -v -r usbcore

modprobe: FATAL: Module usbcore is builtin.
```

Para simular por ejemplo que se desea depurar el uso y gestión de los dispositivos USB, se puede sustituir el fichero usb.c por el siguiente:

https://raw.githubusercontent.com/Torvel/reversing_ucam/master/8.3/code/linux-kernel-4.15_custom/usb.c

Este modulo incluye la impresión extra de mensajes marcados con un identificador especial, que nuestros programas puedan reconocer y parsear: "CUSTOM_DEBUG --"

En ocasiones, los propios modulos ya incluyen por defecto la opción de cargarlos configurándolos en modo debug, lo que hace que su comportamiento sea más "verbose". En el caso por ejemplo de usbstorage, también es posible mediante la opción **USB_STORAGE_DEBUG**. Más info: https://how-to.fandom.com/wiki/How_to_configure_the_Linux_kernel/drivers/usb/storage

7.3. COMPILACIÓN DEL KERNEL

El primer paso para la compilación del kernel, es descargar en el sistema todas aquellas dependencias necesarias para el proceso, es decir, construir el entorno de compilación.

En el caso de la máquina de referencia Ubuntu 18.04, al menos será necesario instalar los siguientes paquetes:

```
# apt-get build-dep linux linux-image-$(uname -r)

# apt-get install libncurses-dev flex bison openssl libssl-dev dkms libelf-dev libudev-dev libpci-dev libiberty-dev autoconf
```

Durante la instalación (especialmente si nunca antes se compilado el kernel en la máquina) se mostrará el error:

E: You must put some 'source' URIs in your sources.list

Esto significa que será necesario añadir nuevos orígenes de software al sistema. Para será necesario modificar /etc/apt/sources.list, (es recomendable hacer un backup) y descomentar o incluid si no lo están, al menos los siguientes orígenes:

deb-src http://es.archive.ubuntu.com/ubuntu bionic main restricted

deb-src http://es.archive.ubuntu.com/ubuntu bionic-updates main restricted

Posteriormente será necesario ejecutar: sudo apt-get update para actualizar los índices del sistema y reintentar la instalación.

7.3.1. Configuración del kernel:

Una vez descargado y modificado el código fuente del kernel a compilar, es posible modificar la configuración del nuevo kernel. En este punto existen distintas opciones de partida sobre las que configurar el nuevo kernel.

- Utilizar como base la configuración actual de la máquina: Es posible hacer una copia de seguridad obteniendo el fichero .config directamente desde el directorio /proc:

```
$ zcat /proc/config.gz > .config
```

- Utilizar directamente la configuración por defecto que recomiende la distribución que se esté utilizando:

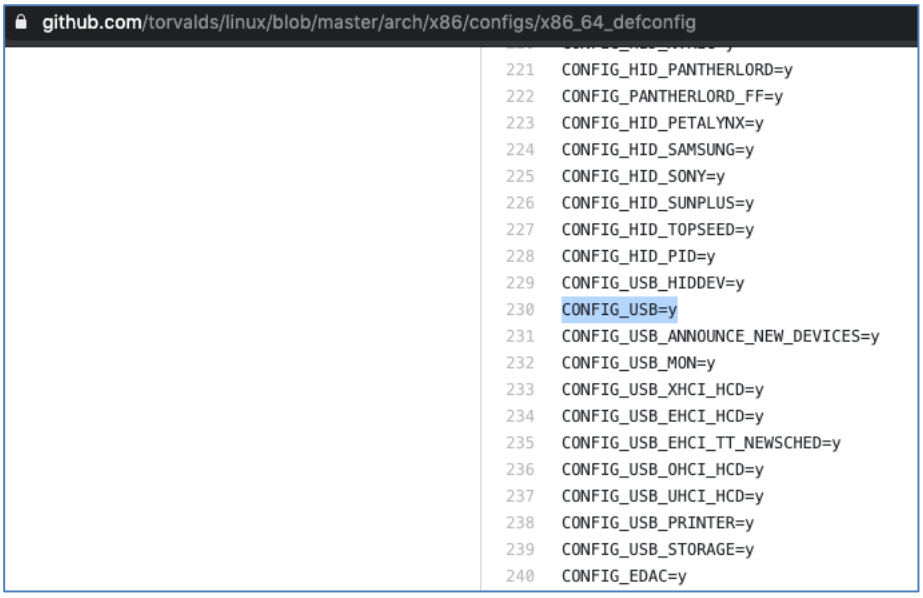
```
cat arch/x86/configs/[i386|x8664]defconfig > .config
```

- Por ultimo, utilizar configuraciones de terceros como por ejemplo descargadas de internet.

Estas configuraciones de partida pueden ser utilizadas como backups si en el proceso de configuración ocurre algún problema inesperado que implique reconfigurar el kernel.

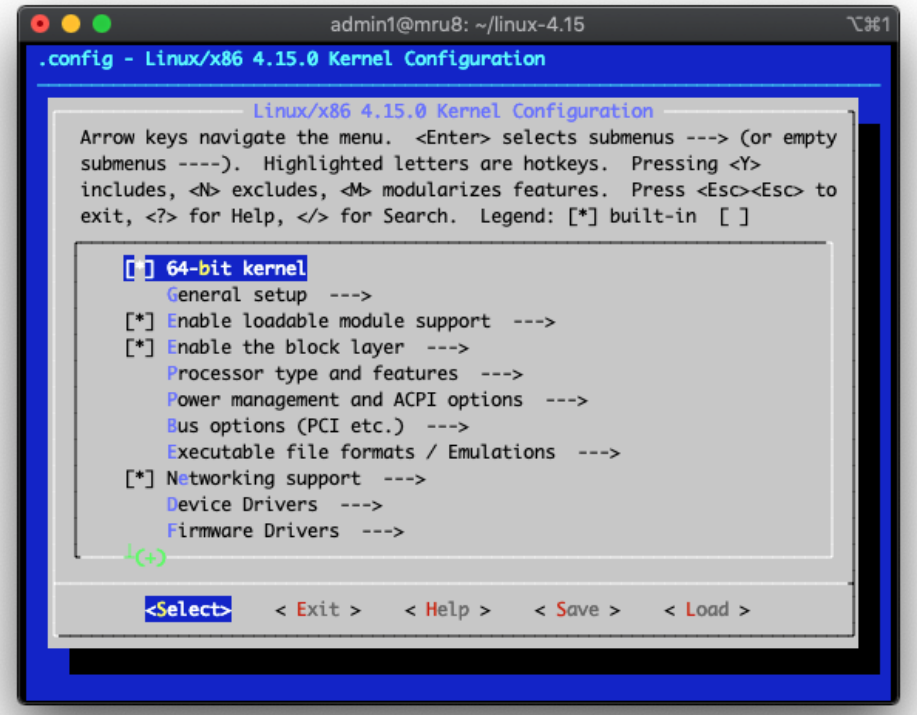
Como se puede ver, en la versión 4.15 del kernel de Linux (https://github.com/torvalds/linux/blob/master/arch/x86/configs/x86_64_defconfig

La configuración de la gestión de los dispositivos USB está incluida como built-in (y):



De cara a configurar el kernel, es posible tanto modificar el fichero de configuración manualmente como hacerlo a través del a herramienta gráfica de configuración. Para lanzar la herramienta de configuración, bastará con ejecutar el siguiente comando en la ruta raíz del directorio que contiene el código fuente.

```
$ make menuconfig
```

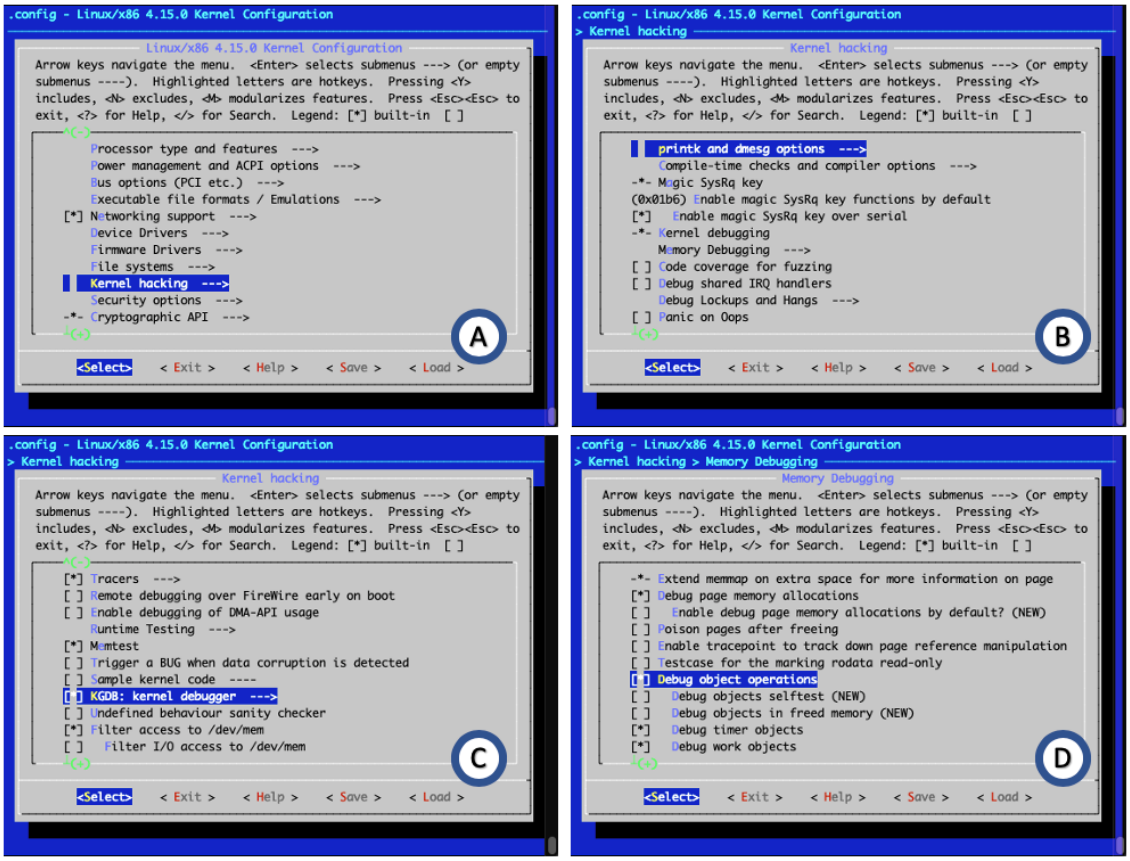


Esto lanzará la herramienta de configuración del kernel Menuconfig, que permite cargar ficheros de configuración previos o generar unos nuevos a partir de una configuración por defecto. Como se puede observar en la siguiente imagen, al tratarse de una primera ejecución, se está obteniendo la configuración por defecto del kernel actual: /boot/config-4.15.0-76-generic. Si se desea rescatar de nuevo, simplemente bastará con ejecutar:

```
cp /boot/config-`uname -r` /usr/src/linux/.config
```

```
admin1@mru8: ~/linux-4.15
admin1@mru8:~/linux-4.15$ make menuconfig
  HOSTCC  scripts/kconfig/mconf.o
  HOSTCC  scripts/kconfig/lxdialog/checklist.o
  HOSTCC  scripts/kconfig/lxdialog/util.o
  HOSTCC  scripts/kconfig/lxdialog/inputbox.o
  HOSTCC  scripts/kconfig/lxdialog/textbox.o
  HOSTCC  scripts/kconfig/lxdialog/yesno.o
  HOSTCC  scripts/kconfig/lxdialog/menubox.o
  HOSTLD  scripts/kconfig/mconf
scripts/kconfig/mconf  Kconfig
#
# using defaults found in /boot/config-4.15.0-76-generic
#


*** End of the configuration.
*** Execute 'make' to start the build or try 'make help'.

admin1@mru8:~/linux-4.15$
```

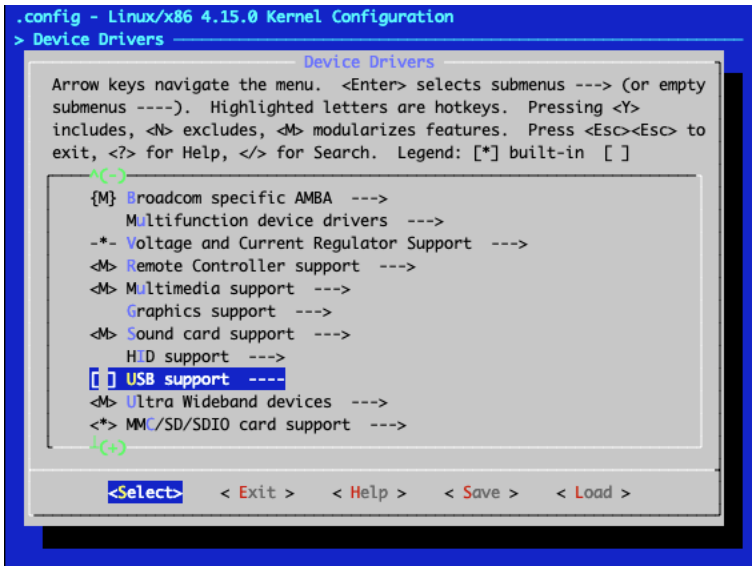
Para ejecutar correctamente la herramienta de configuración, es necesario garantizar que los scripts de configuración sobre los que se basa (script/kconfig/*) tienen permisos de ejecución.



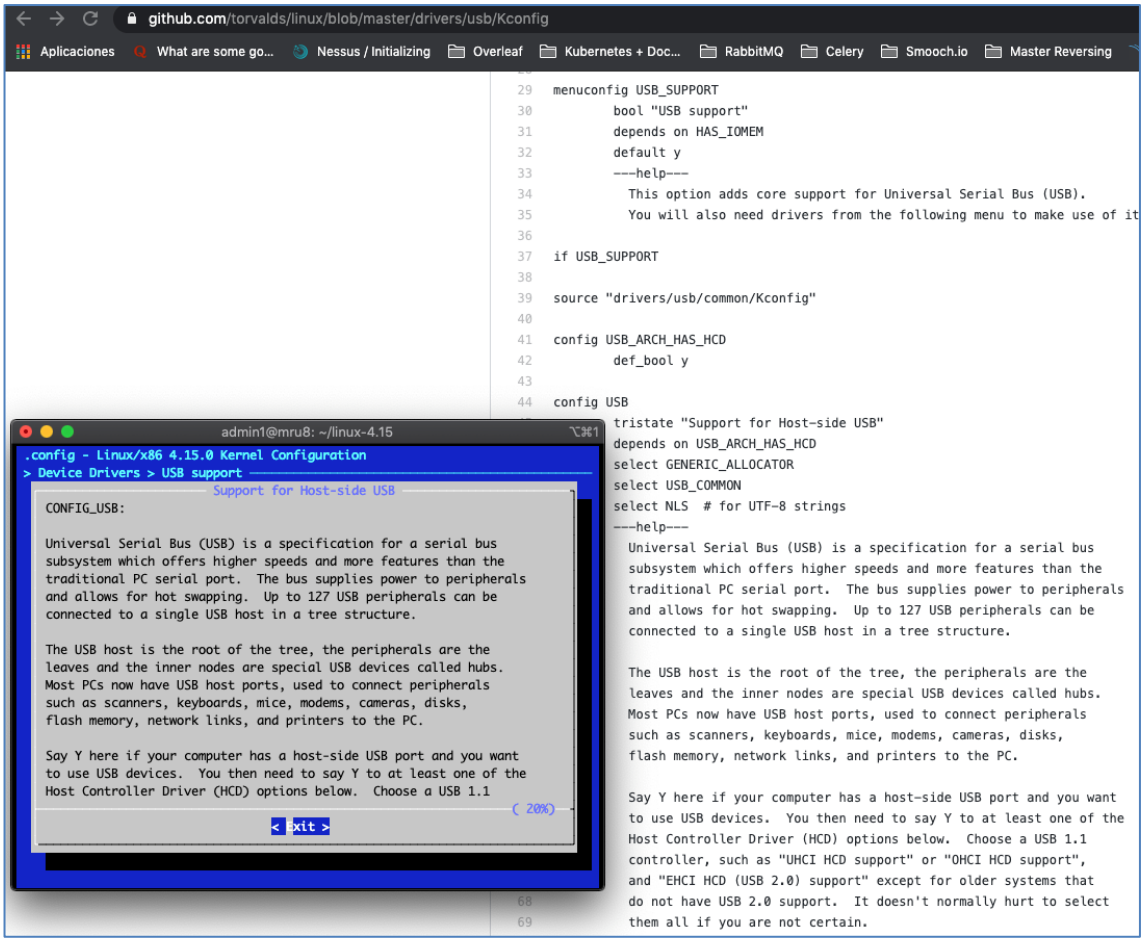
Como se puede observar en la imagen anterior, la herramienta permite configurar a muy bajo nivel qué componentes y características incluir/excluir del kernel, así como habilitar funcionalidades especialmente útiles para depuración como es el caso de las catalogadas dentro de la sección “Kernel hacking” (sección A de la imagen).

La sección Kernel hacking permite activar multitud de funciones diseñadas para depurar el kernel, como por ejemplo las opciones de printk y dmesg (sección B) definiendo entre otras el nivel de log por defecto. También será posible desde esta sección activar el debugger nativo del kernel, KGDB (sección C) o la depuración del uso de la memoria que hace el sistema.

Es posible configurar el modulo “usbcore” directamente desde el configurador del kernel Menuconfig navegando hasta >Device Drivers y a bajo nivel, entrando a > Device Drivers > USB support. En este caso, para poder modificarlo a posteriori, se desmarcará (de [*] a [ ]), modificando su estado de Built-in para tener la posibilidad de modificarlo y cargarlo/descargarlo como módulo.



Como se puede observar, para cada funcionalidad configurable en el configurador del kernel, se encuentra definida por los ficheros **Kconfig** dentro de los directorios de cada módulo que mapean sobre secciones o submenús dentro del configurador.



Es importante notar que, si dejamos fuera del kernel, modulos built-in necesarios para el correcto arranque o funcionamiento del sistema, es posible que no consigamos arrancar, con lo que tendremos que asegurarnos que aunque dejen de estar built-in arranquen al inicio.

Una vez configurado el kernel según nuestras necesidades, solo resta comenzar la compilación. Un proceso que puede ser relativamente lento (puede tomar horas) aunque dependerá de las capacidades HW de la máquina que se emplee.

Paso 1: instalamos kernel-package que nos facilita la compilación

sudo apt install kernel-package

Paso 2: limpiamos el proyecto

fakeroot make-kpkg clean

Paso 3: compilamos y tageamos el kernel custom

make-kpkg --initrd --append-to-version=-1-mru8-custom kernel_image kernel_headers

Para terminar, actualizamos el grub de la máquina y tendremos la opción de seleccionar el nuevo kernel al inicio y usarlo en el arranque.

Una vez arrancado el nuevo kernel, será sencillo buscar trazas de nuestro módulo configurado por ejemplo buscando en los siguientes ficheros:

/var/log/syslog

/var/log/kern.log

Campus Internacional CIBERSEGURIDAD
ENIIT INNOVA IT BUSINESS SCHOOL
UCAM UNIVERSIDAD CATÓLICA DE MURCIA

O mediante la herramienta **dmesg**:

```
admin1@mru8:~$ cat /var/log/syslog
syslog       syslog.1     syslog.2.gz
admin1@mru8:~$ cat /var/log/syslog.1 | grep dispositivo
Feb 28 14:21:33 mru8 kernel: [    0.334345] Nuevo dispositivo USB
admin1@mru8:~$ cat /var/log/kern.log | grep dispositivo
Feb 28 14:21:33 mru8 kernel: [    0.334345] Nuevo dispositivo USB
admin1@mru8:~$ dmesg | grep dispositivo
[    0.334345] Nuevo dispositivo USB
```

7.3.2. Construyendo un kernel custom para depuración

Una opción importante durante la fase de compilación del kernel, es seleccionar que en la vista XX de selección de módulos, marquemos correctamente "**M**" o "**y**", es decir, aquellos que estarán como módulos cargables o Built-in es decir, que no podremos modificar en tiempo de ejecución una vez compilado el kernel. Por esto, será necesario identificar qué módulos deseamos modificar para depurar nuestro sistema y extraerlos del kernel durante la compilación, de este modo podremos modificarlos y recargarlos a posteriori cada vez que lo deseemos en nuestro kernel customizado para depuración.

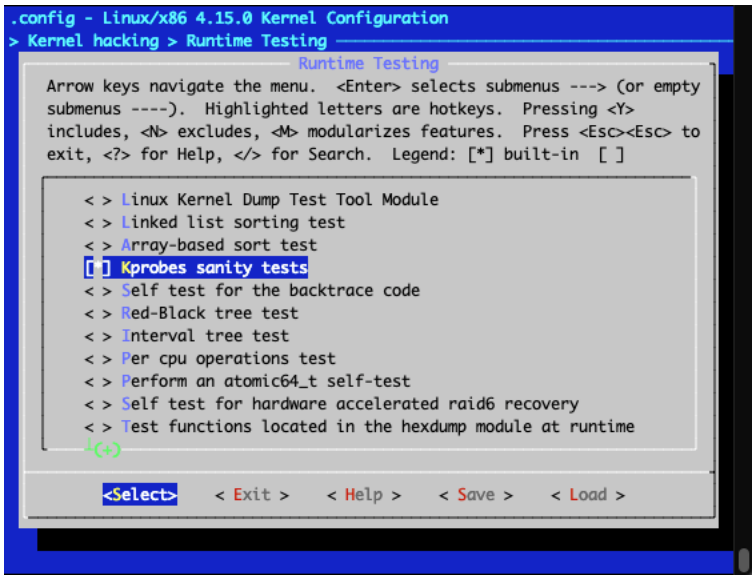
Ya que modificar módulos del kernel conlleva un riesgo alto de error, es conveniente instalar los módulos modificados desde fuera del árbol de directorios por defecto y evitar reemplazar los nativos. De esta forma, si el sistema falla, será recuperable simplemente reiniciando.

7.3.3. Herramientas para la depuración del kernel – Kprobes

Kprobes permite interceptar y depurar módulos y llamadas a sistema del kernel en tiempo de ejecución. Mediante una serie de módulos que el usuario puede crear, configurar e instalar, es posible usar Kprobes para depurar cualquier elemento del kernel, incluyendo aquellos componentes "Built-in" es decir, incluidos dentro del kernel, que solo podríamos modificar mediante la recompilación del kernel.

En el siguiente enlace podemos se encuentra un modulo de ejemplo que utiliza kprobes para interceptar el stack trace y los registros implicados cuando el sistema ejecuta la llamada a sistema do_fork() en el momento de crear nuevos procesos. Esta información es mostrada en el log /var/log/messages

https://github.com/spotify/linux/blob/master/samples/kprobes/kprobe_example.c



Enlaces interesantes:

- Aplicación de parches del kernel: https://www.proware.com.tw/support/software/pdf/linux-patch.pdf

- Guías prácticas sobre kernel hacking: https://www.kernel.org/doc/html/latest/kernel-hacking/index.html

7.4. MODIFICACIÓN DE LA API DEL SISTEMA OPERATIVO. EXFILTRANDO A TRAVÉS DE GLIBC

Interceptar llamadas al sistema o incluso modificar módulos, puede ser una opción muy útil para casos muy concretos en los que a ciencia cierta se conoce que esas llamadas son las que necesitamos para depurar. Sin embargo, si necesitamos depurar a nivel más general (y también más sencillo) la mejor opción es modificar directamente la implementación de las llamadas al sistema operativo que hace glibc, ya que la mayoría de las aplicaciones que corren en el sistema operativo utilizarán esta opción para consumir las syscalls en lugar de implementar sus propias llamadas.

De este modo, modificando únicamente esta librería, podemos añadir todos los prints, logs, etc que necesitemos a cualquier llamada, incluyendo, por ejemplo, el estado de la memoria, parámetros empleados en cada llamada, etc.

De esta misma forma se construyen algunos programas de depuración, protección antimalware y virtualización de muestras. Haciendo mas verbosa la ejecución del sistema operativo, es posible exfiltrar información del kernel al espacio usuario que una aplicación cliente pueda recoger e interpretar.

A continuación, se muestra un ejemplo de cómo construir una versión modificada de **glibc** que imprima mensajes de depuración cuando se realicen llamadas a determinadas syscalls. Posteriormente se usará esta glibc modificada para compilar algunos programas de prueba y observar su comportamiento.

En primer lugar, será necesario obtener el código fuente de glibc que queremos utilizar. En este caso, la que incluye nuestra máquina Ubuntu 18.04 es **glibc-2.28** para ello podemos bajarlo directamente del repositorio git asociado:

```
git clone git://sourceware.org/git/glibc.git
cd glibc
git checkout glibc-2.28
```

Una vez tenemos el código de la versión que nos interesa, podemos modificar las funciones que nos interese. En este caso se modificarán dos especialmente importantes:

- **fopen()**: Invocada cada vez que es necesario el acceso a un fichero de la máquina ya sea para leer o escribir en él.
- **thrd_create():** Invocada cada vez que se pretende crear una nueva hebra para ejecutar una determinada función, por ejemplo mediante posix.

El objetivo por tanto es que el sistema nos muestre un mensaje específico cada vez que un programa compilado con nuestra versión de glibc intente abrir un fichero o crear una hebra.

Para el caso de **fopen()** será necesario editar el fichero **glibc/libio/iofopen.c** y añadir los mensajes necesarios:

```
admin1@mru8: ~
GNU nano 2.9.3                                glibc/libio/iofopen.c

   This exception applies to code released by its copyright holders
   in files containing the exception.  */

#include "libioP.h"
#include <fcntl.h>
#include <stdlib.h>
#include <stddef.h>
#include <shlib-compat.h>
#include <stdio.h>


FILE *
__fopen_maybe_mmap (FILE *fp)
{
#if _G_HAVE_MMAP
  if ((fp->_flags2 & _IO_FLAGS2_MMAP) && (fp->_flags & _IO_NO_WRITES))
    {
      /* Since this is read-only, we might be able to mmap the contents
         directly.  We delay the decision until the first read attempt by
         giving it a jump table containing functions that choose mmap or
         vanilla file operations and reset the jump table accordingly.  */

      if (fp->_mode <= 0)
        _IO_JUMPS_FILE_plus (fp) = &_IO_file_jumps_maybe_mmap;
      else
        _IO_JUMPS_FILE_plus (fp) = &_IO_wfile_jumps_maybe_mmap;
      fp->_wide_data->_wide_vtable = &_IO_wfile_jumps_maybe_mmap;
    }
#endif
  return fp;
}


FILE *
__fopen_internal (const char *filename, const char *mode, int is32)
{

  puts("ejecuta fopen");

  struct locked_FILE
  {
    struct _IO_FILE_plus fp;
#ifdef _IO_MTSAFE_IO
    _IO_lock_t lock;
#endif

^G Get Help    ^O Write Out   ^W Where Is    ^K Cut Text    ^J Justify
^X Exit        ^R Read File   ^\ Replace     ^U Uncut Text  ^T To Spell
```

Importante: Para añadir **prints** y **puts**, será necesario **importar stdio.h**

En el caso de **thrd_create()**, será necesario modificar: **/nptl/thrd_create.c**

```
admin1@mru8: ~
GNU nano 2.9.3                    glibc//nptl/thrd_create.c

/* C11 threads thread creation implementation.
   Copyright (C) 2018 Free Software Foundation, Inc.
   This file is part of the GNU C Library.

   The GNU C Library is free software; you can redistribute it and/or
   modify it under the terms of the GNU Lesser General Public
   License as published by the Free Software Foundation; either
   version 2.1 of the License, or (at your option) any later version.

   The GNU C Library is distributed in the hope that it will be useful,
   but WITHOUT ANY WARRANTY; without even the implied warranty of
   MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.  See the GNU
   Lesser General Public License for more details.

   You should have received a copy of the GNU Lesser General Public
   License along with the GNU C Library; if not, see
   <http://www.gnu.org/licenses/>.  */

#include "thrd_priv.h"
#include <stdio.h>

int
thrd_create (thrd_t *thr, thrd_start_t func, void *arg)
{
  puts("se crea una nueva hebra");

  _Static_assert (sizeof (thr) == sizeof (pthread_t),
                  "sizeof (thr) != sizeof (pthread_t)");

  int err_code = __pthread_create_2_1 (thr, ATTR_C11_THREAD,
                                       (void* (*) (void*))func, arg);
  return thrd_err_map (err_code);
}
```

Una vez realizados los cambios necesarios en la glibc, el siguiente paso será compilarla para poderla utilizar normalmente. Para ello, ejecutaremos los siguientes comandos:

```
export glibc_install="$(pwd)/glibc/build/install"
mkdir build
cd build
../configure --prefix "$glibc_install"
make -j `nproc`
make install -j `nproc`
```

Una vez terminado el último make, nuestra versión de glibc estará compilada y lista para ser usada a su vez en otras compilaciones

```
admin1@mru8:~/glibc/build$ ls
abi-versions.h            gshadow                libnss_db.map       posix                    sunrpc
argp                      hesiod                 libnss_dns.map      pthread-errnos.h         support
assert                    iconv                  libnss_files.map    pthread-errnos.h.d       sysd-rules
bits                      iconvdata              libnss_hesiod.map   pthread-pi-defines.h     sysd-sorted
catgets                   inet                   libnss_nis.map      pthread-pi-defines.h.d   sysd-syscalls
cmath                     intl                   libnss_nisplus.map  pwd                      sysd-versions
config.h                  io                     libpthread.map      resolv                   sysvipc
config.log                jmp_buf-ssp.h          libresolv.map       resource                 tcb-offsets.h
config.make               jmp_buf-ssp.h.d        librt.map           rt                       tcb-offsets.h.d
config.status             ldbl-compat-choose.h   libthread_db.map    rtld-offsets.h           termios
conform                   ld.map                 libutil.map         rtld-offsets.h.d         testrun.sh
cpu-features-offsets.h    libanl.map             link-defines.h      runtime-linker.h         time
cpu-features-offsets.h.d  libBrokenLocale.map    link-defines.h.d    runtime-linker.stamp     timezone
crypt                     libc.a                 linkobj             setjmp                   tlsdesc.h
cstdlib                   libc-abis.h            locale              shadow                   tlsdesc.h.d
csu                       libc-abis.stamp        localedata          shlib.lds                ucontext_i.h
ctype                     libc.map               locale-defines.h    shlib-versions.v         ucontext_i.h.d
debug                     libc-modules.h         locale-defines.h.d  shlib-versions.v.i       unwindbuf.h
dirent                    libc-modules.stmp      login               sigaltstack-offsets.h    unwindbuf.h.d
dlfcn                     libc_nonshared.a       Makefile            sigaltstack-offsets.h.d  Versions.all
dl-tunable-list.h         libc_pic.a             malloc              signal                   Versions.def
dl-tunable-list.stmp      libc_pic.os            manual              socket                   versions.stmp
dummy.c                   libcrypt.map           math                soversions.i             Versions.tmp
dummy.o                   libc.so                mathvec             soversions.mk            Versions.v
dummy.o.dt                libc.so.6              misc                stamp.o                  Versions.v.i
elf                       libdl.map              nis                 stamp.os                 wcsmbs
first-versions.h          libio                  nptl                stamp.oS                 wctype
gmon                      libm.map               nptl_db             stdio-common
gnu                       libmvec.map            nscd                stdlib
gnulib                    libnsl.map             nss                 streams
grp                       libnss_compat.map      po                  string
```

Para probar la detección de creación de hebras, usaremos el siguiente programa de ejemplo (**test_threads.c**), que creará 10 hebras para aumentar un contador:

```c
#define _GNU_SOURCE
#include <assert.h>
#include <gnu/libc-version.h>
#include <stdatomic.h>
#include <stdio.h>
#include <threads.h>

atomic_int acnt;
int cnt;

int f(void* thr_data) {
    for(int n = 0; n < 1000; ++n) {
        ++cnt;
        ++acnt;
    }
    return 0;
}

int main(int argc, char **argv) {
    /* Basic library version check. */
    printf("gnu_get_libc_version() = %s\n", gnu_get_libc_version());

    /* Exercise thrd_create from -pthread,
     * which is not present in glibc 2.27 in Ubuntu 18.04.
     * https://stackoverflow.com/questions/56810/how-do-i-start-threads-in
    thrd_t thr[10];
    for(int n = 0; n < 10; ++n)
        thrd_create(&thr[n], f, NULL);
    for(int n = 0; n < 10; ++n)
        thrd_join(thr[n], NULL);
    printf("The atomic counter is %u\n", acnt);
    printf("The non-atomic counter is %u\n", cnt);
}
```

En el caso de fopen, se usará el programa **test_fopen.c**, que simplemente abrirá un fichero y escribirá un texto en él:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <errno.h>
#include <sys/stat.h>

int main(int argc, char **argv)
{
    FILE * fp;
    fp = fopen ("./test.txt", "w+");
    fprintf(fp, "%s %s %s %d", "We", "are", "in", 2012);
    fclose(fp);
    return(0);
}
```

Una vez creados ambos programas, los compilamos con la ayuda de un pequeño shell script que seteará algunas opciones de compilación en gcc, especialmente la ruta a nuestra versión de glibc mediante el argumento **--rpath**

```
GNU nano 2.9.3          compile_test_fopen.sh          Modified

#!/usr/bin/env bash
set -eux
gcc \
  -L "${glibc_install}/lib" \
  -I "${glibc_install}/include" \
  -Wl,--rpath="${glibc_install}/lib" \
  -Wl,--dynamic-linker="${glibc_install}/lib/ld-linux-x86-64.so.2" \
  -std=c11 \
  -o test_fopen \
  -v \
  test_fopen.c \
  -pthread \
;
```

```
GNU nano 2.9.3          compile_test_threads.sh

#!/usr/bin/env bash
set -eux
gcc \
  -L "${glibc_install}/lib" \
  -I "${glibc_install}/include" \
  -Wl,--rpath="${glibc_install}/lib" \
  -Wl,--dynamic-linker="${glibc_install}/lib/ld-linux-x86-64.so.2" \
  -std=c11 \
  -o test_threads \
  -v \
  test_threads.c \
  -pthread \
;
```

Una vez compilados ambos programas, los ejecutamos para observar la salida:

```
admin1@mru8:~$ ./test_fopen
ejecuta fopen
admin1@mru8:~$ ./test_threads
gnu_get_libc_version() = 2.28
se crea una nueva hebra
se crea una nueva hebra
se crea una nueva hebra
se crea una nueva hebra
se crea una nueva hebra
se crea una nueva hebra
se crea una nueva hebra
se crea una nueva hebra
se crea una nueva hebra
se crea una nueva hebra
The atomic counter is 10000
The non-atomic counter is 10000
```

Conclusión:

Utilizando este mismo procedimiento, es mucho más sencillo aplicar la técnica de "depuración por impresión" a un nivel mucho más global y a la vez menos ruidoso que si editamos puntos concretos del kernel o si leemos todos los mensajes que emite el kernel (por ejemplo leyendo /var/log.kern.log).

Esto nos permitirá, por ejemplo, compilar los programas que se deseen analizar, utilizando una versión custom de glibc y asignar mensajes específicos y fácil de localizar únicamente a los eventos que nos interesan del sistema (a través de las llamadas al kernel que ejecutan). Una opción sería incluso fijar esta versión modificada de glibc a nivel global, por lo que todo programa compilado en nuestro sistema generaría la información deseada.

8. DEPURACIÓN DE CRASH/CORE DUMPS Y SWAP FILES/PARTITIONS

Como se comentó en la unidad anterior, una buena forma de depurar la actividad de la máquina, especialmente cuando se trata de una actuación forense, es a través del análisis de los ficheros de paginación o los de coredump que pueda contener.

Además de lo ya explicado en la unidad 8.2, con la ayuda de GDB corriendo como sudo es posible cargar y depurar Crash/Core dumps de forma similar a como se haría en tiempo de ejecución:

Campus Internacional CIBERSEGURIDAD
ENIIT INNOVA IT BUSINESS SCHOOL
UCAM UNIVERSIDAD CATÓLICA DE MURCIA

```
admin1@mru8: ~
admin1@mru8:~$ gdb -h
This is the GNU debugger.  Usage:

    gdb [options] [executable-file [core-file or process-id]]
    gdb [options] --args executable-file [inferior-arguments ...]

Selection of debuggee and its files:

  --args             Arguments after executable-file are passed to inferior
  --core=COREFILE    Analyze the core dump COREFILE.
  --exec=EXECFILE    Use EXECFILE as the executable.
  --pid=PID          Attach to running process PID.
  --directory=DIR    Search for source files in DIR.
  --se=FILE          Use FILE as symbol file and executable file.
  --symbols=SYMFILE  Read symbols from SYMFILE.
  --readnow          Fully read symbol files on first access.
  --readnever        Do not read symbol files.
  --write            Set writing into executable and core files.

Initial commands and command files:

  --command=FILE, -x Execute GDB commands from FILE.
  --init-command=FILE, -ix
                     Like -x but execute commands before loading inferior.
  --eval-command=COMMAND, -ex
```

Un ejemplo de uso de GDB para encontrar el problema en una herramienta que genera core dumps: https://embeddedbits.org/linux-core-dump-analysis/

Para depurar un proceso y obtener el máximo de información posible, lo ideal es disponer además del fichero coredump de acceso al binario y que idealmente este haya sido compilado con la opción de depuración activada. Una opción más sencilla para obtener una primera aproximación es obtener la salida de **systemd** mediante el uso del comando **journalctl**, que nos dará (aunque de forma global al sistema) una salida similar a la siguiente:

```
admin1@mru8: ~
-- Logs begin at Sun 2020-02-09 13:02:13 UTC, end at Sun 2021-01-31 19:35:51 UTC. --
Feb 09 13:02:13 mru8 kernel: Linux version 4.15.0-76-generic (buildd@lcy01-amd64-029) (gcc version 7.4.0 (Ubuntu 7.4.0-1ubuntu1
Feb 09 13:02:13 mru8 kernel: Command line: BOOT_IMAGE=/boot/vmlinuz-4.15.0-76-generic root=UUID=dd483b47-4cd8-431f-822d-66e6cc8
Feb 09 13:02:13 mru8 kernel: KERNEL supported cpus:
Feb 09 13:02:13 mru8 kernel:   Intel GenuineIntel
Feb 09 13:02:13 mru8 kernel:   AMD AuthenticAMD
Feb 09 13:02:13 mru8 kernel:   Centaur CentaurHauls
Feb 09 13:02:13 mru8 kernel: Disabled fast string operations
Feb 09 13:02:13 mru8 kernel: x86/fpu: Supporting XSAVE feature 0x001: 'x87 floating point registers'
Feb 09 13:02:13 mru8 kernel: x86/fpu: Supporting XSAVE feature 0x002: 'SSE registers'
Feb 09 13:02:13 mru8 kernel: x86/fpu: Supporting XSAVE feature 0x004: 'AVX registers'
Feb 09 13:02:13 mru8 kernel: x86/fpu: Supporting XSAVE feature 0x008: 'MPX bounds registers'
Feb 09 13:02:13 mru8 kernel: x86/fpu: Supporting XSAVE feature 0x010: 'MPX CSR'
Feb 09 13:02:13 mru8 kernel: x86/fpu: xstate_offset[2]:  576, xstate_sizes[2]:  256
Feb 09 13:02:13 mru8 kernel: x86/fpu: xstate_offset[3]:  832, xstate_sizes[3]:   64
Feb 09 13:02:13 mru8 kernel: x86/fpu: xstate_offset[4]:  896, xstate_sizes[4]:   64
Feb 09 13:02:13 mru8 kernel: x86/fpu: Enabled xstate features 0x1f, context size is 960 bytes, using 'compacted' format.
Feb 09 13:02:13 mru8 kernel: e820: BIOS-provided physical RAM map:
Feb 09 13:02:13 mru8 kernel: BIOS-e820: [mem 0x0000000000000000-0x000000000009e7ff] usable
Feb 09 13:02:13 mru8 kernel: BIOS-e820: [mem 0x000000000009e800-0x000000000009ffff] reserved
Feb 09 13:02:13 mru8 kernel: BIOS-e820: [mem 0x00000000000dc000-0x00000000000fffff] reserved
Feb 09 13:02:13 mru8 kernel: BIOS-e820: [mem 0x0000000000100000-0x000000003fedffff] usable
Feb 09 13:02:13 mru8 kernel: BIOS-e820: [mem 0x000000003fee0000-0x000000003fefefff] ACPI data
Feb 09 13:02:13 mru8 kernel: BIOS-e820: [mem 0x000000003feff000-0x000000003feffffff] ACPI NVS
lines 1-24
```

BIBLIOGRAFÍA

- Guía completa de programación de módulos para el kernel de Linux https://tldp.org/LDP/lkmpg/2.6/html/c38.html
- Diferentes métodos para "Debugging by printing" del kernel: https://elinux.org/Debugging_by_printing#Disabling_printk_messages_at_compile_time
- Consejos útiles a la hora de depurar el kernel: https://elinux.org/Kernel_Debugging_Tips
- Web útil para buscar directamente cadenas de texto dentro del código fuente de Linux: https://livegrep.com/search/linux
- Modificando módulos del kernel para hookear llamadas al sistema: https://www.youtube.com/watch?v=0IQlpFWTFbM&t=3s
- Apuntes rápidos para comenzar con la depuración de coredumps mediante GDB: https://stackoverflow.com/a/22711917/13152261